杂项

Morning Flower and Evening Oath

XCPC vector 模板分册

建设稿：验证状态与适用边界见各条目及仓库 verification/。

目录

使用约定 iii

第一章 Dancing Links 1
- 1.1 精确覆盖（Dancing Links） . . . 1
- 1.2 最少行重复覆盖（Dancing Links） . . . 5

第二章 数位 DP 10
- 2.1 数位 DP：十进制数码计数 . . . 10

第三章 插头 DP 12
- 3.1 插头 DP：网格单闭环计数 . . . 12
- 3.2 插头 DP：恰好 K 个不嵌套回路 . . . 16

第四章 DP 优化 20
- 4.1 斜率优化 . . . 20
  - 4.1.1 单调斜率凸包：最小值与决策 . . . 20
- 4.2 四边形不等式优化 . . . 22
  - 4.2.1 分治决策单调性：一层分组 DP . . . 22
- 4.3 WQS 二分 . . . 25
  - 4.3.1 WQS 二分：至多 k 个不相邻位置 . . . 25

第五章 离线算法 27
- 5.1 CDQ 分治 . . . 27
  - 5.1.1 四维偏序：双层 CDQ . . . 27
  - 5.1.2 静态三维偏序计数 . . . 32
  - 5.1.3 三维最长链、计数与方案 . . . 34
- 5.2 整体二分 . . . 39
  - 5.2.1 整体二分：动态区间第 k 小 . . . 39
- 5.3 莫队算法 . . . 43
  - 5.3.1 普通莫队算法 . . . 43
    - 5.3.1.1 普通莫队：静态区间统计 . . . 43
  - 5.3.2 带修改莫队 . . . 45
    - 5.3.2.1 带修改的莫队 . . . 45
  - 5.3.3 树上莫队 . . . 48
    - 5.3.3.1 树上莫队：点权与边权 . . . 48
  - 5.3.4 回滚莫队 . . . 52
    - 5.3.4.1 回滚莫队 . . . 52
  - 5.3.5 莫队二次离线 . . . 56
    - 5.3.5.1 莫队二次离线：异或位数配对 . . . 56

第六章 配套知识与验证范围 59
- 6.1 带修莫队：时间轴与区间状态 . . . 59
- 6.2 莫队二次离线：前缀事件与自身扣除 . . . 60

第一部分 跨类依赖 **61**

附录 A 数论 **62**
A.1 模算术简介 . . . 62
A.1.1 静态模整数 . . . 62
A.2 卢卡斯定理 . . . 64
A.2.1 Lucas：小素数模的大下标组合数 . . . 64

附录 B 组合数学 **66**
B.1 排列组合 . . . 66
B.1.1 阶乘组合数 . . . 66

附录 C 树状数组 **68**
C.1 树状数组第 k 小与组合依赖 . . . 68

附录 D 线段树 **70**
D.1 线段树基础 . . . 70
D.1.1 通用懒标记线段树与边界搜索 . . . 70
D.1.2 通用线段树与左右边界搜索 . . . 75
D.1.3 线段树：区间乘、加、赋值与模意义求和 . . . 80

附录 E 树上问题 **83**
E.1 树链剖分 . . . 83
E.1.1 重链剖分与 LCA . . . 83

使用约定

C++20，默认包含 bits/stdc++.h 与 cassert，并使用 std 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 vector 代码。

第一章　Dancing Links

1.1　精确覆盖（Dancing Links）

ExactCover(m,rows)：列 0..m-1，各稀疏行内列号互异、无需排序；行可重复或为空，保留原 0-based 行号。solve() 返回 optional<vector<int»，无解为 nullopt，m=0 为有值的空解。只求任意首解，不保证最少行数。成功/失败均恢复链与列计数，可重复调用及复制；实现字段不可外改，移后源对象仅销毁或重新赋值。m 和行数 <INT_MAX，1+m+ 非零数 ≤INT_MAX。构造 O(行数 +m+ 非零数)，空间 O(m+ 非零数)，递归深度 ≤m，搜索最坏指数。

```
struct ExactCover
{
    struct Node { int l, r, u, d, c, row; };
    int m;
    vector<Node> a;
    vector<int> s, ans;

    ExactCover(int columns, const vector<vector<int>> &rows) : m(columns)
    {
        assert(0 <= m && m < INT_MAX && rows.size() < INT_MAX);
        size_t z = size_t(m) + 1;
        for (const auto &r : rows)
        {
            assert(r.size() <= size_t(INT_MAX) - z);
            z += r.size();
        }
        a.reserve(z);
        s.assign(m + 1, 0);
        for (int c = 0; c <= m; c++)
            a.push_back({c ? c - 1 : m, c == m ? 0 : c + 1, c, c, c, -1});
```

稀疏行建链与删除（接上页同一结构体）：

```
        vector<int> seen(m, -1);
        for (int i = 0; i < int(rows.size()); i++)
        {
            int h = -1;
            for (int col : rows[i])
            {
                assert(0 <= col && col < m && seen[col] != i);
                seen[col] = i;
                int c = col + 1, x = a.size(), u = a[c].u;
                a.push_back({x, x, u, c, c, i});
                a[u].d = a[c].u = x;
                s[c]++;
                if (h == -1) h = x;
                else
                {
                    int l = a[h].l;
                    a[x].l = l; a[x].r = h;
                    a[l].r = a[h].l = x;
                }
            }
        }
    }

    void cover(int c)
    {
        a[a[c].l].r = a[c].r;
        a[a[c].r].l = a[c].l;
        for (int i = a[c].d; i != c; i = a[i].d)
            for (int j = a[i].r; j != i; j = a[j].r)
            {
                a[a[j].u].d = a[j].d;
                a[a[j].d].u = a[j].u;
                s[a[j].c]--;
            }
    }
```

逆序恢复与搜索（接上页同一结构体）：

```
    void uncover(int c)
    {
        for (int i = a[c].u; i != c; i = a[i].u)
            for (int j = a[i].l; j != i; j = a[j].l)
            {
                s[a[j].c]++;
                a[a[j].u].d = a[a[j].d].u = j;
            }
        a[a[c].l].r = a[a[c].r].l = c;
    }

    bool dfs()
    {
        int c = a[0].r;
        if (!c) return true;
        for (int j = a[c].r; j; j = a[j].r)
            if (s[j] < s[c]) c = j;
        cover(c);
        for (int i = a[c].d; i != c; i = a[i].d)
        {
            ans.push_back(a[i].row);
            for (int j = a[i].r; j != i; j = a[j].r) cover(a[j].c);
            bool ok = dfs();
            for (int j = a[i].l; j != i; j = a[j].l) uncover(a[j].c);
            if (ok)
            {
                uncover(c);
                return true;
            }
            ans.pop_back();
        }
        uncover(c);
        return false;
    }

    optional<vector<int>> solve()
    {
        ans.clear();
        ans.reserve(m);
        if (!dfs()) return nullopt;
        return ans;
    }
};
```

模板题使用：Luogu P4929 / 精确覆盖

题意：读入 N×M 的 01 矩阵，N,M≤500 且非零数 ≤5000；每列恰被选择行覆盖一次。稀疏建模后输出原 1-based 行号，任一可行解均可；无解输出 No Solution!。

所需模板：ExactCover（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4929

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> rows(n);
    for (int i = 0; i < n; i++)
        for (int j = 0, x; j < m; j++)
        {
            cin >> x;
            if (x) rows[i].push_back(j);
        }
    ExactCover dlx(m, rows);
    auto ans = dlx.solve();
    if (!ans) cout << "No Solution!\n";
    else
    {
        for (int i : *ans) cout << i + 1 << ' ';
        cout << '\n';
    }
}
```

1.2 最少行重复覆盖（Dancing Links）

MinimumCover(m, rows)，列号 0..m-1，每行内部互异，允许空行/重复行。solve() 返回最少行数的一组原 0-based 行号；nullopt 无解，有值空向量表示 m=0。每列至少一次，允许重叠；不含权值、覆盖次数或全解枚举。0≤m<INT_MAX，行数 <INT_MAX，m+1+ 非零数 ≤INT_MAX；前提由调用方保证。构造 O(行数 +m+ 非零数)，空间 O(m+ 非零数)；递归搜索最坏指数、深度 ≤m，每状态 O(m+ 非零数)。成功失败均恢复链，可重复求解或复制；重建用重新赋值。字段及辅助方法不作为可修改接口，移动后源对象只销毁或赋值。下界选取不能被同一行共同覆盖的列，至少还需同样多行；按最少候选列搜索，逆序恢复只改左右链。

```
struct MinimumCover
{
    struct Node
    {
        int l, r, u, d, c, row;
    };
    int m, best = 0;
    vector<Node> a;
    vector<int> s, path, ans;
    vector<unsigned char> mark;

    MinimumCover(int columns, const vector<vector<int>> &rows) : m(columns)
    {
        assert(0 <= m && m < INT_MAX && rows.size() < INT_MAX);
        size_t z = size_t(m) + 1;
        for (const auto &r : rows)
        {
            assert(r.size() <= size_t(INT_MAX) - z);
            z += r.size();
        }
        a.reserve(z);
        s.assign(m + 1, 0);
        mark.resize(m + 1);
        path.reserve(m);
        ans.reserve(m);
        for (int c = 0; c <= m; c++)
            a.push_back({c ? c - 1 : m, c == m ? 0 : c + 1, c, c, c, -1});
```

稀疏建链与删列（接上页同一结构体）：

```
        vector<int> seen(m, -1);
        for (int i = 0; i < int(rows.size()); i++)
        {
            int h = -1;
            for (int col : rows[i])
            {
                assert(0 <= col && col < m && seen[col] != i);
                seen[col] = i;
                int c = col + 1, x = a.size(), u = a[c].u;
                a.push_back({x, x, u, c, c, i});
                a[u].d = x;
                a[c].u = x;
                s[c]++;
                if (h == -1) h = x;
                else
                {
                    int l = a[h].l;
                    a[x].l = l;
                    a[x].r = h;
                    a[l].r = x;
                    a[h].l = x;
                }
            }
        }
    }

    // x is a selected row node, not a column header.
    void remove(int x)
    {
        for (int i = a[x].d; i != x; i = a[i].d)
        {
            a[a[i].l].r = a[i].r;
            a[a[i].r].l = a[i].l;
        }
    }
```

逆序恢复与可采纳下界（接上页同一结构体）：

```
    void restore(int x)
    {
        for (int i = a[x].u; i != x; i = a[i].u)
        {
            a[a[i].l].r = i;
            a[a[i].r].l = i;
        }
    }

    int lower_bound()
    {
        fill(mark.begin(), mark.end(), 0);
        for (int c = a[0].r; c; c = a[c].r) mark[c] = 1;
        int bound = 0;
        for (int c = a[0].r; c; c = a[c].r)
        {
            if (!mark[c]) continue;
            bound++;
            mark[c] = 0;
            for (int i = a[c].d; i != c; i = a[i].d)
                for (int j = a[i].r; j != i; j = a[j].r)
                    mark[a[j].c] = 0;
        }
        return bound;
    }
```

最优搜索与求解入口（接上页同一结构体）：

```
    void dfs()
    {
        int depth = path.size(), c = a[0].r;
        if (!c)
        {
            if (depth < best)
            {
                best = depth;
                ans = path;
            }
            return;
        }
        if (lower_bound() >= best - depth) return;
        for (int j = a[c].r; j; j = a[j].r)
            if (s[j] < s[c]) c = j;
        for (int i = a[c].d; i != c; i = a[i].d)
        {
            path.push_back(a[i].row);
            remove(i);
            for (int j = a[i].r; j != i; j = a[j].r) remove(j);
            dfs();
            for (int j = a[i].l; j != i; j = a[j].l) restore(j);
            restore(i);
            path.pop_back();
        }
    }

    optional<vector<int>> solve()
    {
        path.clear();
        ans.clear();
        best = m + 1;
        dfs();
        if (best > m) return nullopt;
        return ans;
    }
};
```

接口用法演示：最少行重复覆盖（本地接口示例）

演示约定：读入 n 个稀疏集合和 m 列；每行先给元素数，再给互异的 0-based 列号。输出最少选行数及原 0-based 行号，使每列至少出现一次；无解输出-1，m=0 输出 0 与空方案。此为接口演示，不冒充线上模板题。

所需模板：MinimumCover（第 5 页，1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/search/dlx/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> rows(n);
    for (auto &row : rows)
    {
        int k;
        cin >> k;
        row.resize(k);
        for (int &c : row) cin >> c;
    }
    MinimumCover cover(m, rows);
    auto ans = cover.solve();
    if (!ans) cout << -1 << '\n';
    else
    {
        cout << ans->size() << '\n';
        for (int row : *ans) cout << row << ' ';
        cout << '\n';
    }
}
```

第二章　数位 DP

2.1　数位 DP：十进制数码计数

```cpp
array<__int128, 10> digit_count(long long n)
{
    struct State
    {
        __int128 ways = 0;
        array<__int128, 10> count{};
    };
    if (n <= 0) return {};
    string s = to_string(n);
    vector<array<State, 2>> memo(s.size());
    vector<array<bool, 2>> seen(s.size());
    auto dfs = [&](auto &&self, int pos, bool tight, bool started) -> State
    {
        if (pos == (int)s.size())
        {
            State result;
            result.ways = started;
            return result;
        }
        if (!tight && seen[pos][started]) return memo[pos][started];
        State result;
        int limit = tight ? s[pos] - '0' : 9;
        for (int d = 0; d <= limit; d++)
        {
            bool next = started || d != 0;
            State child = self(self, pos + 1, tight && d == limit, next);
            result.ways += child.ways;
            for (int x = 0; x < 10; x++)
                result.count[x] += child.count[x];
            if (next) result.count[d] += child.ways;
        }
        if (!tight)
        {
            seen[pos][started] = true;
            memo[pos][started] = result;
        }
        return result;
    };
    return dfs(dfs, 0, true, false).count;
}
```

digit_count(n) 返回正整数 1..n 中十种十进制数码的出现次数；n≤0 返回全零，整数 0 本身不计入。支持 signed long long 的正数全范围，返回 array<__int128,10>，不可在完整范围下擅自转成 64 位。区间 [a,b]（1≤a≤b）用两次前缀结果相减。

递归状态 pos、tight、started：tight 表示前缀仍贴上界；started 为假时前导零不计数，终点全前导零不算一个正整数。State 同时累计方案数 ways 和数码次数 count；选定实际数码 d 后增加 child.ways。只记忆化非 tight 状态，每次调用独立创建 memo/seen，可反复处理不同上界。L 为位数，时间 O(100L)、空间 O(10L)、递归深度 O(L)，L≤19。P2602 的 b≤10^12，答案小于 13×10^12，可转 long long 输出；更大上界需 128 位输出。P2602 当前代码已 AC，10 个测试点，单点 4ms；排名未核查。

应用题补充：P2602 [ZJOI2010] 数字计数

题意：输入 1≤a≤b≤10^12，输出区间中 0..9 的出现次数；前缀相减，前导零不计，两个上界独立记忆化。该题范围内每个计数可转 long long。

所需模板：digit_count（第 10 页，2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2602

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long a, b;
    cin >> a >> b;
    auto left = digit_count(a - 1);
    auto right = digit_count(b);
    for (int d = 0; d < 10; d++)
        cout << (long long)(right[d] - left[d]) << " \n"[d == 9];
}
```

第三章　插头 DP

3.1　插头 DP：网格单闭环计数

grid_cycle<Count>(a) 统计覆盖所有可用格子的单个无向简单环；矩形网格中. 必须经过、* 不能经过，仅上下左右相邻。空网格/全障碍返回 0。自动转置到较窄方向，宽度 m≤15，尺寸须可用 int 表示。状态每槽 4 位，同编号表示同一路径的两个端点，按首次出现规范化；闭环只允许发生在最后一个可用格子且无其他插头时。

Count 支持 0/1 构造和 +=；固定整数必须保证所有中间计数不溢出，可改模整数或 cpp_int。S 为最多可达状态数，固定宽度计数的期望时间 O(n*m^2*S)、状态表 O(S)，另有网格 O(nm)。S 随宽度指数增长；cpp_int 还需计入位复杂度。

```
template <class Count>
Count grid_cycle(vector<string> a)
{
    int n = a.size();
    if (n == 0) return Count(0);
    int m = a[0].size();
    for (auto &row : a)
    {
        assert((int)row.size() == m);
        for (char c : row) assert(c == '.' || c == '*');
    }
    if (m == 0) return Count(0);
    if (n < m)
    {
        vector<string> b(m, string(n, '*'));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) b[j][i] = a[i][j];
        a.swap(b);
        swap(n, m);
    }
    assert(m <= 15);
    pair<int, int> last = {-1, -1};
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (a[i][j] == '.') last = {i, j};
    using Key = unsigned long long;
    unordered_map<Key, Count> dp, next;
    dp[0] = Count(1);
    Count answer = 0;
    for (int i = 0; i < n; i++)
    {
        next.clear();
        for (auto &[s, ways] : dp) next[s << 4] += ways;
        dp.swap(next);
```

轮廓转移（接上页同一函数）：

```
        for (int j = 0; j < m; j++)
        {
            next.clear();
            bool down = i + 1 < n && a[i + 1][j] == '.';
            bool right = j + 1 < m && a[i][j + 1] == '.';
            for (auto &[s, ways] : dp)
            {
                vector<int> p(m + 1);
                for (int k = 0; k <= m; k++) p[k] = s >> (4 * k) & 15;
                int x = p[j];
                int y = p[j + 1];
                p[j] = 0;
                p[j + 1] = 0;
                auto add = [&]()
                {
                    array<int, 16> id{};
                    int count = 0;
                    Key key = 0;
                    for (int k = 0; k <= m; k++)
                    {
                        int v = p[k];
                        if (v && !id[v]) id[v] = ++count;
                        key |= Key(id[v]) << (4 * k);
                    }
                    next[key] += ways;
                };
                if (a[i][j] == '*')
                {
                    if (!x && !y) add();
                }
                else if (!x && !y)
                {
                    if (down && right)
                    {
                        p[j] = 15;
                        p[j + 1] = 15;
                        add();
                    }
                }
```

延伸、合并与最终闭环（接上页同一函数）：

```
                else if (!x || !y)
                {
                    if (down)
                    {
                        p[j] = x + y;
                        add();
                        p[j] = 0;
                    }
                    if (right)
                    {
                        p[j + 1] = x + y;
                        add();
                    }
                }
                else if (x != y)
                {
                    for (int &v : p)
                        if (v == y) v = x;
                    add();
                }
                else if (make_pair(i, j) == last)
                {
                    if (all_of(p.begin(), p.end(), [](int v)
                    {
                        return v == 0;
                    })) answer += ways;
                }
            }
            dp.swap(next);
        }
    }
    return answer;
}
```

轮廓与不变量 槽 j、j+1 是当前格的左、上插头，转移后成为下、右插头。处理过的可用点度数固定为 2，每个未闭合连通块是一条路径，在轮廓上恰有两个同编号端点。行首左移 4 位；右边界插头恒为 0，所以移位不丢有效信息。最多 8 条路径，临时编号 15 不会与规范编号冲突。

转移与去重 无入边时必须创建下、右两条出边；单入边选择延伸方向；不同编号的双入边合并路径并统一编号；相同编号表示闭环，只能在最后一个可用点且其他槽全零时接收。按首次出现重编号使相同连通关系唯一。每个边集唯一对应逐格选择，不按起点或方向重复计数。

数值与验证 固定整数要保证所有中间计数可表示，不能只检查最后答案。P5056 仅内部 (n-1)(m-1) 个格子可能二选一，其余转移至多一个。所有中间计数不超过 2 的 121 次方，因此下面用 128 位整数仍能保证精确计数。已穷举 74954 个小障碍网格并与独立递归环搜索核对；完整 2 至 14 阶偶数方格与 OEIS A003763 核对。输入超过题目范围时状态数仍可能很大。

模板题使用：P5056 插头 DP

题意：2≤n,m≤12，所有点格必须形成唯一闭环。仅同时存在下、右邻格时可能二选一，至多 $(12\text{-}1)^2$=121 次，因此全部中间计数 ≤2^121，用 ___int128 安全；十进制输出。

所需模板：grid_cycle（第 12 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5056

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<string> a(n);
    for (auto &row : a) cin >> row;
    __int128 answer = grid_cycle<__int128>(a);
    string out;
    do
    {
        out += char('0' + answer % 10);
        answer /= 10;
    } while (answer);
    reverse(out.begin(), out.end());
    cout << out << '\n';
}
```

3.2 插头 DP：恰好 K 个不嵌套回路

grid_cycles<Count>(a,k,nested=false) 统计覆盖全部. 格子的恰好 k 个无向简单环；* 为障碍，只有上下左右相邻。默认禁止任意环包含另一个环；nested=true 允许嵌套。无可用格且 k=0 时返回 1，其余无法覆盖时返回 0。k 为非负 int 且 k<INT_MAX；矩形尺寸可用 int 表示，自动转置后宽度 m≤15。

同编号的两个插头表示同一路径端点，规范化编号。闭环时环数加 1；禁止嵌套时检查当前格左侧插头数为偶数。每个环在其最后扫描顶点闭合，该奇偶性是环的嵌套深度奇偶；若存在嵌套，必有深度 1 的环被拒绝。只在扫完全部格子、无剩余插头时取答案。

Count 需支持 0/1 构造、默认构造为 0、复制和 +=；所有中间计数均须可表示，可用模整数或 cpp_int。S 为固定环数下最多轮廓状态数，固定宽度计数下期望时间 $O(n*m^2*(k+1)*(S+1))$，两层状态表 $O((k+1)*(S+1))$，网格 $O(nm)$。S 随宽度指数增长；高精度计数另计位复杂度。

```
template <class Count>
Count grid_cycles(vector<string> a, int k, bool nested = false)
{
    assert(k >= 0);
    int n = a.size();
    if (n == 0) return Count(k == 0);
    int m = a[0].size();
    long long cells = 0;
    for (auto &row : a)
    {
        assert((int)row.size() == m);
        for (char c : row)
        {
            assert(c == '.' || c == '*');
            cells += c == '.';
        }
    }
    if (cells == 0) return Count(k == 0);
    if (k == 0 || k > cells / 4) return Count(0);
    if (n < m)
    {
        vector<string> b(m, string(n, '*'));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) b[j][i] = a[i][j];
        a.swap(b);
        swap(n, m);
    }
    assert(m <= 15);
    using Key = unsigned long long;
    vector<unordered_map<Key, Count>> dp(k + 1), next(k + 1);
    dp[0][0] = Count(1);
```

行移位、规范化与创建路径（接上页同一函数）：

```
    for (int i = 0; i < n; i++)
    {
        for (int c = 0; c <= k; c++)
        {
            next[c].clear();
            for (auto &[s, ways] : dp[c]) next[c][s << 4] += ways;
        }
        dp.swap(next);
        for (int j = 0; j < m; j++)
        {
            for (auto &t : next) t.clear();
            bool down = i + 1 < n && a[i + 1][j] == '.';
            bool right = j + 1 < m && a[i][j + 1] == '.';
            for (int c = 0; c <= k; c++)
            {
                for (auto &[s, ways] : dp[c])
                {
                    array<int, 16> p{};
                    for (int t = 0; t <= m; t++) p[t] = s >> (4 * t) & 15;
                    int x = p[j];
                    int y = p[j + 1];
                    p[j] = 0;
                    p[j + 1] = 0;
                    auto add = [&](int count)
                    {
                        array<int, 16> id{};
                        int top = 0;
                        Key key = 0;
                        for (int t = 0; t <= m; t++)
                        {
                            int v = p[t];
                            if (v && !id[v]) id[v] = ++top;
                            key |= Key(id[v]) << (4 * t);
                        }
                        next[count][key] += ways;
                    };
                    if (a[i][j] == '*')
                    {
                        if (!x && !y) add(c);
                    }
                    else if (!x && !y)
                    {
                        if (down && right)
                        {
                            p[j] = 15;
                            p[j + 1] = 15;
                            add(c);
                        }
                    }
```

延伸、合并与闭环计数（接上页同一函数）：

```
                    else if (!x || !y)
                    {
                        if (down)
                        {
                            p[j] = x + y;
                            add(c);
                            p[j] = 0;
                        }
                        if (right)
                        {
                            p[j + 1] = x + y;
                            add(c);
                        }
                    }
                    else if (x != y)
                    {
                        for (int &v : p)
                            if (v == y) v = x;
                        add(c);
                    }
                    else if (c < k)
                    {
                        int parity = 0;
                        for (int t = 0; t < j; t++) parity ^= p[t] != 0;
                        if (nested || parity == 0) add(c + 1);
                    }
                }
            }
            dp.swap(next);
        }
    }
    auto it = dp[k].find(0);
    return it == dp[k].end() ? Count(0) : it->second;
}
```

应用题补充：HDU4285 circuits

题意：官方竞赛应用：多组 2≤n,m≤12 网格，恰好 K 个互不嵌套回路覆盖所有可用格，答案模 10^9+7。官方样例为 2、6；不是单环模板的旧 AC。当前最小抄写版 41022698 已 AC，11216MS、27236K。

所需模板：grid_cycles（第 16 页，3.2 节）、ModInt（第 62 页，A.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=4285

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        int n, m, k;
        cin >> n >> m >> k;
        vector<string> a(n);
        for (auto &row : a) cin >> row;
        cout << grid_cycles<ModInt<1000000007>>(a, k).v << '\n';
    }
}
```

接口用法演示：多环覆盖与嵌套开关（精确计数）

演示约定：输入 n,m,k,nested 与网格，nested=0 禁止嵌套，非 0 允许嵌套；使用 cpp_int 避免中间计数溢出，需要 Boost 头文件。空网格 k=0 有一个空方案。此驱动为接口演示。

所需模板：grid_cycles（第 16 页，3.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

额外依赖（不属于标准头文件）：`boost/multiprecision/cpp_int.hpp`，须由编译环境提供并显式包含。

来源：参考文档

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k, nested;
    cin >> n >> m >> k >> nested;
    vector<string> a(n);
    for (auto &row : a) cin >> row;
    using Count = boost::multiprecision::cpp_int;
    cout << grid_cycles<Count>(a, k, nested != 0) << '\n';
}
```

第四章　DP 优化

4.1　斜率优化

4.1.1　单调斜率凸包：最小值与决策

默认空；add(m,b,id) 按斜率非增加入直线 mx+b，query(x) 按 x 非减查询，返回 (pair<int128,int>)(最小值, 编号)。首次查询前至少加一条线；可交替加线与查询。并列保留最早加入的直线，不是最小 id；同斜率只留最小截距，截距也同则留旧线。q/head 仅供只读；查询会永久淘汰旧候选，不支持回退 x 或任意斜率。要求 |m|,|b|,|x|<=10^18，总加线数 <INT_MAX。先提升 int128 再做差与交叉乘法，范围内精确无浮点；返回值勿未经证明窄化为 long long。总 M 次加线、Q 次查询 O(M+Q)，空间 O(M)，被查询淘汰的前缀仍留在 vector 中。

```
struct MonotoneHull
{
    using ll = long long;
    using I = __int128_t;
    struct Line
    {
        ll m, b;
        int id;

        I value(ll x) const
        {
            return I(m) * x + b;
        }
    };
    vector<Line> q;
    int head = 0;
    ll last_x = 0;
    bool queried = false;

    void add(ll m, ll b, int id)
    {
        assert(q.empty() || m <= q.back().m);
        if (int(q.size()) > head && q.back().m == m)
        {
            if (q.back().b <= b) return;
            q.pop_back();
        }
        Line c{m, b, id};
        while (int(q.size()) - head >= 2)
        {
            auto a = q[q.size() - 2];
            auto b = q.back();
            if ((I(b.b) - a.b) * (b.m - I(c.m)) <
                (I(c.b) - b.b) * (a.m - I(b.m))) break;
            q.pop_back();
        }
        q.push_back(c);
    }
```

查询（接上页同一结构体）：

```
    pair<I, int> query(ll x)
    {
        assert(int(q.size()) > head);
        assert(!queried || last_x <= x);
        queried = true;
        last_x = x;
        while (head + 1 < int(q.size()) && q[head + 1].value(x) < q[head].value(x))
        {
            head++;
        }
        return {q[head].value(x), q[head].id};
    }
};
```

应用题补充：P2365 任务安排

题意：IOI2002 任务安排：保持任务顺序分批，最小化完成时刻乘费用系数的总和。n<=5000，s<=50，t 和 f 在 1..100。前缀 T/C，dp[i]=T[i]*C[i]+s*C[n]+min(dp[j]-(T[i]+s)*C[j])；斜率和查询方向均满足单调，答案及截距在 long long 范围内。

所需模板：MonotoneHull（第 20 页，4.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2365

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long s;
    cin >> n >> s;
    vector<long long> t(n + 1), c(n + 1);
    for (int i = 1; i <= n; i++)
    {
        cin >> t[i] >> c[i];
        t[i] += t[i - 1];
        c[i] += c[i - 1];
    }
    MonotoneHull hull;
    hull.add(0, 0, 0);
    long long dp = 0;
    for (int i = 1; i <= n; i++)
    {
        dp = hull.query(t[i] + s).first + t[i] * c[i] + s * c[n];
        hull.add(-c[i], dp, i);
    }
    cout << dp << '\n';
}
```

4.2 四边形不等式优化

4.2.1 分治决策单调性：一层分组 DP

monotone_dp_layer(prev,first,cost,opt) 返回 cur，prev 下标 0..n，1<=first<=n<INT_MAX。对 i=first..n 求 min(prev[j]+cost(j+1,i))，j 在 first-1..i-1，cost 是闭区间代价；用于第 g 组时 first=g。inf=LLONG_MAX/4；prev[first-1] 必须可达，其他 inf 跳过，cost 对全部合法区间有限。所有有限 prev、cost 及转移和须严格在 (-inf,inf)，保证 long long 加法安全。cur[0..first-1]=inf，opt 对应为-1；其余 opt 返回最小最优 j。调用方须证明该并列规则下 opt[i] 非减，本函数不会检查，任意 DP 不能套用。cost 须无副作用且 O(1) 才能得 O(n log(n+1)) 单层时间；递归深度 O(log n)，输出与辅助 O(n)。

```
template<class Cost>
vector<long long> monotone_dp_layer(const vector<long long> &prev, int first,
                                    Cost cost, vector<int> &opt)
{
    using ll = long long;
    constexpr ll inf = LLONG_MAX / 4;
    int n = int(prev.size()) - 1;
    assert(1 <= first && first <= n && prev[first - 1] != inf);
    vector<ll> cur(n + 1, inf);
    opt.assign(n + 1, -1);
    auto solve = [&](auto &&self, int l, int r, int lo, int hi) -> void
    {
        if (l > r) return;
        int mid = l + (r - l) / 2;
        for (int j = lo; j <= min(hi, mid - 1); j++)
        {
            if (prev[j] == inf) continue;
            ll value = prev[j] + cost(j + 1, mid);
            if (value < cur[mid])
            {
                cur[mid] = value;
                opt[mid] = j;
            }
        }
        assert(opt[mid] != -1);
        self(self, l, mid - 1, lo, opt[mid]);
        self(self, mid + 1, r, opt[mid], hi);
    };
    solve(solve, first, n, first - 1, n - 1);
    return cur;
}
```

应用题补充：CF321E Ciel and Gondolas

题意：Ciel and Gondolas：n<=4000，k<=800，非负对称矩阵对角 0；恰分 k 个非空连续组，最小化组内无序对权和。二维前缀和 O(1) 求闭区间代价；四边形不等式保证最小最优决策非减。

所需模板：monotone_dp_layer（第 22 页，4.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/CF321E

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<vector<int>> sum(n + 1, vector<int>(n + 1));
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            char c;
            cin >> c;
            sum[i][j] = c - '0' + sum[i - 1][j] + sum[i][j - 1] - sum[i - 1][j - 1];
        }
    }
    auto cost = [&](int l, int r) -> long long
    {
        return (sum[r][r] - sum[l - 1][r] - sum[r][l - 1] + sum[l - 1][l - 1]) / 2;
    };
    vector<long long> dp(n + 1, LLONG_MAX / 4);
    vector<int> opt;
    dp[0] = 0;
    for (int g = 1; g <= k; g++)
    {
        dp = monotone_dp_layer(dp, g, cost, opt);
    }
    cout << dp[n] << '\n';
}
```

应用题补充：P4767 [IOI 2000] 邮局加强版

题意：n<=3000 个村庄，k<=300 个邮局，坐标 1..10000；最小化到最近邮局距离之和。先排序；每个连续组在中位数处建邮局，前缀和 O(1) 求组代价。该代价满足四边形不等式，使用恰分 k 组 DP。

所需模板：monotone_dp_layer（第 22 页，4.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4767

```cpp
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<long long> x(n + 1), sum(n + 1);
    for (int i = 1; i <= n; i++) cin >> x[i];
    sort(x.begin() + 1, x.end());
    for (int i = 1; i <= n; i++) sum[i] = sum[i - 1] + x[i];
    auto cost = [&](int l, int r) -> long long
    {
        int mid = (l + r) / 2;
        return x[mid] * (mid - l + 1) - (sum[mid] - sum[l - 1]) +
               (sum[r] - sum[mid]) - x[mid] * (r - mid);
    };
    vector<long long> dp(n + 1, LLONG_MAX / 4);
    vector<int> opt;
    dp[0] = 0;
    for (int g = 1; g <= k; g++)
    {
        dp = monotone_dp_layer(dp, g, cost, opt);
    }
    cout << dp[n] << '\n';
}
```

4.3 WQS 二分

4.3.1 WQS 二分：至多 k 个不相邻位置

wqs_independent_set(a,k) 返回 0-based 数组中至多选 k 个两两不相邻位置的最大权和；允许空选、空数组、负权，k>=0。n=a.size()<INT_MAX，W=max|a[i]|，要求 n*W<=LLONG_MAX/4（以足够宽类型核验），因此不含 LLONG_MIN。只返回最优值，不返回方案。路径独立集等价于路径边匹配，定基数最优值离散凹；每次选择扣 penalty，pair 比较先权和、并列取更多位置。若罚 0 的最优数量 <=k 直接返回；否则二分仍能选至少 k 个的最大非负整数罚值，答案为惩罚后最优值 +penalty*k。不是任意 DP 的 WQS 通用封装。O(n log(W+2)) 时间、O(1) 额外空间。

```
long long wqs_independent_set(const vector<long long> &a, int k)
{
    using ll = long long;
    assert(k >= 0);
    if (k == 0 || a.empty()) return 0;
    auto solve = [&](ll penalty) -> pair<ll, int>
    {
        pair<ll, int> prev{0, 0}, cur{0, 0};
        for (ll w : a)
        {
            pair<ll, int> take{prev.first + w - penalty, prev.second + 1};
            prev = cur;
            cur = max(cur, take);
        }
        return cur;
    };
    auto free = solve(0);
    if (free.second <= k) return free.first;
    ll lo = 0, hi = max(0LL, *max_element(a.begin(), a.end()));
    while (lo < hi)
    {
        ll mid = lo + (hi - lo + 1) / 2;
        if (solve(mid).second >= k) lo = mid;
        else hi = mid - 1;
    }
    return solve(lo).first + lo * k;
}
```

应用题补充：P1484 种树（WQS）

题意：种树：至多选 k 个不相邻坑，最大获利；允许负数。n<=300000，k<=n/2，|a[i]|<=10^6。WQS 的凹性证明依赖路径匹配模型。

所需模板：wqs_independent_set（第 25 页，4.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1484

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    cout << wqs_independent_set(a, k) << '\n';
}
```

第五章　离线算法

5.1　CDQ 分治

5.1.1　四维偏序：双层 CDQ

Dominance4D::count(p, strict=false) 返回 vector<int>，第 i 项为其他点 j 在四个坐标均 ≤p[i] 的数量；strict=true 改为四个坐标均严格小于。按原输入顺序返回，排除自身，重复点按不同输入编号计数；空输入返回空。坐标可为任意 signed64，输入不修改。n≤INT_MAX/2；每点答案 ≤n-1，总点对数须用 long long 汇总。

每点拆成更新与查询两个事件；x、y 同坐标时，弱模式更新先于查询，严格模式查询先于更新。外层 CDQ 固定 x 序，只抽取左半更新与右半查询；内层按 y 序分治，以 z 归并并用 Fenwick 统计 w 前缀。严格模式比较 z< 且查询 w 排名-1；弱模式 z≤、查询完整排名，最后减去自身一次。递归 CDQ，不模拟 DFS 栈。时间 O(n $\log^3$(n+1))，额外空间 O(n)，两层递归总调用深度 O(log(n+1))。坐标只比较，不做减一或取负，避免极值溢出；w 排名减一是压缩后的安全下标。

坐标压缩与 x 序事件：

```
struct Dominance4D
{
    static vector<int> count(const vector<array<long long, 4>> &p, bool strict = false)
    {
        assert(p.size() <= INT_MAX / 2);
        int n = p.size();
        vector<int> ans(n), bit(n + 1);
        vector<long long> coords;
        for (auto point : p) coords.push_back(point[3]);
        sort(coords.begin(), coords.end());
        coords.erase(unique(coords.begin(), coords.end()), coords.end());
        struct Event
        {
            int id, w;
            bool query;
        };
        vector<Event> events;
        for (int i = 0; i < n; i++)
        {
            int w = lower_bound(coords.begin(), coords.end(), p[i][3]) - coords.begin() + 1;
            events.push_back({i, w, false});
            events.push_back({i, w, true});
        }
        auto less = [&](const Event &a, const Event &b, int axis)
        {
            if (p[a.id][axis] != p[b.id][axis])
                return p[a.id][axis] < p[b.id][axis];
            return strict ? a.query > b.query : a.query < b.query;
        };
        sort(events.begin(), events.end(), [&](const Event &a, const Event &b)
        {
            return less(a, b, 0);
        });
```

Fenwick 与外层 CDQ（接上页同一函数）：

```
        auto add = [&](int x, int value)
        {
            for (; x <= n; x += x & -x) bit[x] += value;
        };
        auto sum = [&](int x)
        {
            int result = 0;
            for (; x; x -= x & -x) result += bit[x];
            return result;
        };
        auto outer = [&](auto &&self, int l, int r) -> void
        {
            if (r - l <= 1) return;
            int mid = l + (r - l) / 2;
            self(self, l, mid);
            self(self, mid, r);
            vector<Event> a;
            for (int i = l; i < mid; i++)
                if (!events[i].query) a.push_back(events[i]);
            for (int i = mid; i < r; i++)
                if (events[i].query) a.push_back(events[i]);
            sort(a.begin(), a.end(), [&](const Event &u, const Event &v)
            {
                return less(u, v, 1);
            });
            vector<Event> temp(a.size());
```

两次唯一分离 每个合法更新–查询对在 x 事件序中有先后顺序，恰好在外层递归的一处分到左右两半。只提取左更新和右查询后，再按 y 排序并递归，恰好在内层的一处分离。z 双指针与 w 前缀和检查余下两维，因此既不漏计也不重复计数。

相等坐标 x、y 相等时，严格模式将查询放在更新前面，阻止这一对进入贡献方向；弱模式反过来。z 的比较和 w 的前缀下标也必须一起切换。弱模式每个点的自身恰被算一次，最后减一；重复点仍按不同编号保留。

内层 CDQ、回滚与结果（接上页同一函数）：

```
            auto inner = [&](auto &&solve, int left, int right) -> void
            {
                if (right - left <= 1) return;
                int middle = left + (right - left) / 2;
                solve(solve, left, middle);
                solve(solve, middle, right);
                int i = left;
                for (int j = middle; j < right; j++)
                {
                    while (i < middle && (strict ? p[a[i].id][2] < p[a[j].id][2]
                                                 : p[a[i].id][2] <= p[a[j].id][2]))
                    {
                        if (!a[i].query) add(a[i].w, 1);
                        i++;
                    }
                    if (a[j].query) ans[a[j].id] += sum(a[j].w - strict);
                }
                for (int j = left; j < i; j++)
                    if (!a[j].query) add(a[j].w, -1);
                merge(a.begin() + left, a.begin() + middle,
                      a.begin() + middle, a.begin() + right, temp.begin() + left,
                      [&](const Event &u, const Event &v)
                      {
                          return p[u.id][2] < p[v.id][2];
                      });
                copy(temp.begin() + left, temp.begin() + right, a.begin() + left);
            };
            inner(inner, 0, a.size());
        };
        outer(outer, 0, events.size());
        if (!strict)
            for (int &value : ans) value--;
        return ans;
    }
};
```

模板题使用：QOJ21566 四维偏序

题意：四个坐标各为 1..n 的排列，统计四维全部严格小于的有向点对数；64 位汇总。

所需模板：Dominance4D（第 27 页，5.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://qoj.ac/problem/21566

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<array<long long, 4>> p(n);
    for (auto &point : p)
        for (auto &x : point) cin >> x;
    auto count = Dominance4D::count(p, true);
    cout << accumulate(count.begin(), count.end(), 0LL) << '\n';
}
```

接口用法演示：API 四维逐点支配计数

演示约定：输入 n 和 strict(0/1)，再输入 n 个四维点；按原顺序逐行输出每点答案，排除自身。

所需模板：Dominance4D（第 27 页，5.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://qoj.ac/problem/21566

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    bool strict;
    cin >> n >> strict;
    vector<array<long long, 4>> p(n);
    for (auto &point : p)
        for (auto &x : point) cin >> x;
    auto count = Dominance4D::count(p, strict);
    for (int value : count) cout << value << '\n';
}
```

5.1.2 静态三维偏序计数

代码依赖：Fenwick（第 68 页，C.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

count(points) 返回每个原输入位置被其他点逐坐标 ≤ 的次数，重复坐标仍是不同点，排除自身。输入 array<long long,3>，完整 signed64 坐标，空输入返回空；输入不变。内部按三维字典序合并重合点，压缩第三维，CDQ 归并与 Fenwick 回滚。O(n $\log^2$(n+1)) 时间、O(n) 空间、O(log(n+1)) 递归；n 须 ≤INT_MAX/2 且内存可分配，保证共享 Fenwick 的 int 步进不溢出。仅静态无权弱偏序，不含严格不等号、动态更新或 CDQ 递推。

```
struct Dominance3D
{
    static vector<int> count(const vector<array<long long, 3>> &p)
    {
        assert(p.size() <= INT_MAX / 2);
        int n = int(p.size());
        vector<int> order(n), group(n), w;
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int i, int j) { return p[i] < p[j]; });
        vector<array<long long, 3>> v;
        for (int i : order)
        {
            if (v.empty() || v.back() != p[i])
            {
                v.push_back(p[i]);
                w.push_back(0);
            }
            ++w.back();
            group[i] = int(v.size()) - 1;
        }
        int m = int(v.size());
        vector<long long> z;
        for (auto q : v) z.push_back(q[2]);
        sort(z.begin(), z.end());
        z.erase(unique(z.begin(), z.end()), z.end());
        struct Node
        {
            long long y;
            int z, w, id;
        };
        vector<Node> a(m), tmp(m);
        vector<int> cnt(m), ans(n);
        for (int i = 0; i < m; ++i)
            a[i] = {v[i][1], int(lower_bound(z.begin(), z.end(), v[i][2]) -
                                 z.begin()) + 1, w[i], i};
        Fenwick<int> bit(int(z.size()));
```

三维偏序的 CDQ 归并与原序返回（接上页同一 count 函数）：

```
        auto cdq = [&](auto &&self, int l, int r) -> void
        {
            if (r - l <= 1) return;
            int mid = l + (r - l) / 2;
            self(self, l, mid);
            self(self, mid, r);
            int i = l, k = l;
            for (int j = mid; j < r; ++j)
            {
                while (i < mid && a[i].y <= a[j].y)
                {
                    bit.add(a[i].z, a[i].w);
                    tmp[k++] = a[i++];
                }
                cnt[a[j].id] += bit.sum(a[j].z);
                tmp[k++] = a[j];
            }
            for (int t = l; t < i; ++t) bit.add(a[t].z, -a[t].w);
            while (i < mid) tmp[k++] = a[i++];
            copy(tmp.begin() + l, tmp.begin() + r, a.begin() + l);
        };
        cdq(cdq, 0, m);
        for (int i = 0; i < n; ++i)
            ans[i] = cnt[group[i]] + w[group[i]] - 1;
        return ans;
    }
};
```

模板题使用：Luogu P3810 【模板】三维偏序 / 陌上花开

题意：1≤n≤100000，1≤ 三个坐标 ≤k≤200000。对每个输入点统计其他点三个坐标均 ≤ 自身的次数，输出次数为 0..n−1 的点数直方图。重复位置分别计数且各自排除自己。k 只约束题面坐标，库内部压缩，无需外部初始化；空输入和 signed64 极值另属 API 扩展。

所需模板：Fenwick（第 68 页，C.1 节）、Dominance3D（第 32 页，5.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3810

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;
    vector<array<long long, 3>> p(n);
    for (auto &[x, y, z] : p) cin >> x >> y >> z;
    vector<int> hist(n);
    for (int x : Dominance3D::count(p)) ++hist[x];
    for (int x : hist) cout << x << '\n';
}
```

5.1.3 三维最长链、计数与方案

Chain3D(points, mod, strict=true)，坐标 array<long long,3>，输入不变。

mod 为 1..INT_MAX，n≤INT_MAX/2 且内存可分配。

strict=true 要求每一步三个坐标都严格增加；false 为三坐标不减，并固定按 (x,y,z, 原输入编号) 递增的规范顺序，重合点可依编号串联，不把相同坐标的排列重复计数。length/count 为最长链长度/模方案数；空输入为 0 和 1%mod。len/ways/parent 按 0-based 原输入编号存每点结尾的最优状态与前驱，无前驱为-1；path() 返回一条最长链原编号。并列时逐步选编号较小的前驱/终点，不保证整条路径字典序最小；计数为 0 模 mod 不表示无方案。CDQ 左递归 → 转移 → 右递归，严格模式整组相同 x 不互转；y 扫描、z 压缩树状数组维护最大长度和同长计数。$O(n \log^2(n+1))$ 时间、O(n) 空间、O(log(n+1)) 递归；坐标只比较，支持完整 signed64。所有成员只读，重新赋值可重算。

```
struct Chain3D
{
    using Point = array<long long, 3>;

    struct State
    {
        int len = 0, ways = 0, end = -1;
    };

    vector<int> len, ways, parent;
    int length = 0, count = 0, last = -1;

    Chain3D(const vector<Point> &p, int mod, bool strict = true)
    {
        assert(mod > 0 && p.size() <= INT_MAX / 2);
        int n = p.size();
        count = 1 % mod;
        len.assign(n, 1);
        ways.assign(n, 1 % mod);
        parent.assign(n, -1);
        if (!n) return;
```

坐标顺序、分组与状态合并（接上页同一结构体）：

```
        vector<int> order(n), z(n), cut{0};
        vector<long long> values;
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int x, int y)
        {
            if (p[x] != p[y]) return p[x] < p[y];
            return x < y;
        });
        for (int i = 1; i < n; i++)
        {
            if (!strict || p[order[i]][0] != p[order[i - 1]][0]) cut.push_back(i);
        }
        cut.push_back(n);
        for (auto q : p) values.push_back(q[2]);
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        for (int i = 0; i < n; i++)
        {
            z[i] = lower_bound(values.begin(), values.end(), p[i][2]) - values.begin() + 1;
        }
        vector<State> bit(values.size() + 1);
        auto join = [&](State x, State y)
        {
            if (x.len != y.len) return x.len > y.len ? x : y;
            x.ways = ((long long)x.ways + y.ways) % mod;
            x.end = min(x.end, y.end);
            return x;
        };
```

先转移再右递归（同一构造函数内）（接上页同一结构体）：

```
        auto cdq = [&](auto &&self, int l, int r) -> void
        {
            if (r - l <= 1) return;
            int mid = l + (r - l) / 2;
            self(self, l, mid);
            {
                vector<int> a(order.begin() + cut[l], order.begin() + cut[mid]);
                vector<int> b(order.begin() + cut[mid], order.begin() + cut[r]);
                auto by_y = [&](int x, int y)
                {
                    return p[x][1] < p[y][1];
                };
                sort(a.begin(), a.end(), by_y);
                sort(b.begin(), b.end(), by_y);
                int i = 0;
                for (int v : b)
                {
                    while (i < (int)a.size())
                    {
                        int u = a[i];
                        if (strict ? p[u][1] >= p[v][1] : p[u][1] > p[v][1]) break;
                        i++;
                        for (int t = z[u]; t < (int)bit.size(); t += t & -t)
                        {
                            bit[t] = join(bit[t], {len[u], ways[u], u});
                        }
                    }
                    State best;
                    for (int t = z[v] - strict; t > 0; t -= t & -t)
                    {
                        best = join(best, bit[t]);
                    }
                    if (!best.len) continue;
                    if (best.len + 1 > len[v])
                    {
                        len[v] = best.len + 1;
                        ways[v] = best.ways;
                        parent[v] = best.end;
                    }
                    else if (best.len + 1 == len[v])
                    {
                        ways[v] = ((long long)ways[v] + best.ways) % mod;
                        parent[v] = min(parent[v], best.end);
                    }
                }
                for (int j = 0; j < i; j++)
                {
                    for (int t = z[a[j]]; t < (int)bit.size(); t += t & -t) bit[t] = State{};
                }
            }
            self(self, mid, r);
        };
```

汇总答案与路径恢复（接上页同一结构体）：

```
        cdq(cdq, 0, cut.size() - 1);
        State best;
        for (int i = 0; i < n; i++) best = join(best, {len[i], ways[i], i});
        length = best.len;
        count = best.ways;
        last = best.end;
    }

    vector<int> path() const
    {
        vector<int> ans;
        for (int u = last; u != -1; u = parent[u]) ans.push_back(u);
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

接口用法演示：三维最长链与原编号方案（接口演示）

演示约定：输入 n,mod,strict 及 n 个三维整数点。strict=1 为三维都严格增加；strict=0 为三维不减，重合点按原编号串联，不重复计算它们的排列。输出全局长度/模方案数、一条原编号路径，以及各原输入点的结尾长度/模方案数/前驱编号。空输入全局为 0 和 1%mod，路径为空；mod=1 时计数 0 也能恢复路径。

所需模板：Chain3D（第 34 页，5.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://oi-wiki.org/misc/cdq-divide/

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, mod, strict;
    cin >> n >> mod >> strict;
    vector<Chain3D::Point> p(n);
    for (auto &v : p) cin >> v[0] >> v[1] >> v[2];
    Chain3D chain(p, mod, strict);
    cout << chain.length << ' ' << chain.count << '\n';
    for (int id : chain.path()) cout << id << ' ';
    cout << '\n';
    for (int i = 0; i < n; i++)
    {
        cout << chain.len[i] << ' ' << chain.ways[i] << ' ' << chain.parent[i] << '\n';
    }
}
```

应用题补充：HDU4742 Pinball Game 3D

题意：T≤3 组，每组 1≤n≤10^5 个互不重合的三维点，坐标 0..2^30。命中顺序要求三维均不减，按命中点集计数；输出最长长度及方案数模 2^30。调用 Chain3D 的弱偏序模式；无重合点使每个可行点集只有一个规范顺序。5 秒、32768K，当前最小抄写版已 AC。严格模式、重合点及其他模数仍属本地验证范围。

所需模板：Chain3D（第 34 页，5.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://acm.hdu.edu.cn/showproblem.php?pid=4742

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<Chain3D::Point> p(n);
        for (auto &v : p) cin >> v[0] >> v[1] >> v[2];
        Chain3D chain(p, 1 << 30, false);
        cout << chain.length << ' ' << chain.count << '\n';
    }
}
```

5.2 整体二分

5.2.1 整体二分：动态区间第 k 小

代码依赖：Fenwick（第 68 页，C.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

OverallKth(a) 读取 0-based 非空初始数组；后续位置用 1..n。set(pos,value) 登记单点赋值；query(l,r,k) 登记闭区间第 k 小并返回 0-based 查询 ID，1<=k<=r-l+1。run() 按查询 ID 返回 vector<long long>，答案对应登记查询时的状态，不是最后数组；可重复 run，也可继续登记后重新 run。支持完整 long long 值域，不做值的算术；同值修改跳过。依赖 Fenwick<int>，递归值域分治，保持时间顺序并撤销贡献。n<=INT_MAX/2，事件数 E=n+2U+Q<=INT_MAX（U 为实际变值次数），另须满足内存限制。V 为出现过的不同值数，时间 O(E log(E+1)+E log(n+1) log(V+1))，空间 O(n+E)。只读 current/events/queries，不外部修改。

```
struct OverallKth
{
    using ll = long long;

    struct Event
    {
        int l, r, k, id;
        ll value;
    };

    int n, queries = 0;
    vector<ll> current;
    vector<Event> events;

    OverallKth(const vector<ll> &a) : n(a.size()), current(a)
    {
        assert(n > 0);
        for (int i = 1; i <= n; i++)
        {
            events.push_back({i, 0, 1, -1, a[i - 1]});
        }
    }

    void set(int pos, ll value)
    {
        assert(1 <= pos && pos <= n);
        if (current[pos - 1] == value) return;
        events.push_back({pos, 0, -1, -1, current[pos - 1]});
        events.push_back({pos, 0, 1, -1, value});
        current[pos - 1] = value;
    }

    int query(int l, int r, int k)
    {
        assert(1 <= l && l <= r && r <= n);
        assert(1 <= k && k <= r - l + 1);
        int id = queries++;
        events.push_back({l, r, k, id, 0});
        return id;
    }
```

继续求解（接上页同一结构体）：

```
    vector<ll> run() const
    {
        vector<ll> answer(queries), values;
        if (queries == 0) return answer;
        auto a = events;
        for (auto e : a)
        {
            if (e.id < 0) values.push_back(e.value);
        }
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end());
        for (auto &e : a)
        {
            if (e.id < 0)
            {
                e.value = lower_bound(values.begin(), values.end(), e.value) - values.begin();
            }
        }
        vector<Event> buffer(a.size());
        vector<bool> left(a.size());
        Fenwick<int> bit(n);
```

继续求解（接上页同一结构体）：

```
        auto solve = [&](auto &&self, int lo, int hi, int begin, int end) -> void
        {
            if (begin == end) return;
            if (lo == hi)
            {
                for (int i = begin; i < end; i++)
                {
                    if (a[i].id >= 0) answer[a[i].id] = values[lo];
                }
                return;
            }
            int mid = lo + (hi - lo) / 2, count = 0;
            for (int i = begin; i < end; i++)
            {
                auto &e = a[i];
                if (e.id < 0)
                {
                    left[i] = e.value <= mid;
                    if (left[i]) bit.add(e.l, e.k);
                }
                else
                {
                    int smaller = bit.query(e.l, e.r);
                    left[i] = e.k <= smaller;
                    if (!left[i]) e.k -= smaller;
                }
                count += left[i];
            }
            int p = begin, q = begin + count;
            for (int i = begin; i < end; i++)
            {
                auto e = a[i];
                if (e.id < 0 && left[i]) bit.add(e.l, -e.k);
                if (left[i]) buffer[p++] = e;
                else buffer[q++] = e;
            }
            copy(buffer.begin() + begin, buffer.begin() + end, a.begin() + begin);
            self(self, lo, mid, begin, begin + count);
            self(self, mid + 1, hi, begin + count, end);
        };
        solve(solve, 0, int(values.size()) - 1, 0, a.size());
        return answer;
    }
};
```

应用题补充：P2617 Dynamic Rankings（整体二分）

题意：n,m<=100000，初值和修改值 0..1e9；Q l r k 查询闭区间第 k 小，C pos value 单点赋值。先按原时间顺序登记，最后统一求解输出；不继承在线 DynamicKth 的 AC。

所需模板：Fenwick（第 68 页，C.1 节）、OverallKth（第 39 页，5.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2617

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    OverallKth solver(a);
    for (int i = 0; i < m; i++)
    {
        char op;
        cin >> op;
        if (op == 'Q')
        {
            int l, r, k;
            cin >> l >> r >> k;
            solver.query(l, r, k);
        }
        else
        {
            int pos;
            long long value;
            cin >> pos >> value;
            solver.set(pos, value);
        }
    }
    for (long long value : solver.run()) cout << value << '\n';
}
```

5.3 莫队算法

5.3.1 普通莫队算法

5.3.1.1 普通莫队：静态区间统计

```
struct Mo
{
    struct Query
    {
        int l, r, id;
    };
    int n;
    vector<Query> q;

    Mo(int n) : n(n)
    {
        assert(n >= 0);
    }
    int add(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        int id = q.size();
        q.push_back({l, r, id});
        return id;
    }
    template <class Add, class Del, class Ans>
    void run(Add add, Del del, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block;
            int y = b.l / block;
            if (x != y) return x < y;
            return x & 1 ? a.r > b.r : a.r < b.r;
        });
        int l = 0;
        int r = 0;
        for (auto a : ord)
        {
            while (l > a.l) add(--l);
            while (r < a.r) add(r++);
            while (l < a.l) del(l++);
            while (r > a.r) del(--r);
            ans(a.id);
        }
        while (l < r) del(l++);
    }
};
```

Mo(n)，n≥0；add(l,r) 登记 0-based 半开区间并返回查询编号，允许空区间。run(add,del,ans,block=0) 中加删回调接收下标，答案回调接收原查询编号。开始时统计状态须为空，结束后删除剩余点恢复空状态；可重复 run，运行期间不能修改查询表。适用于加删任意位置语义相同的集合统计；逆序对等顺序敏感统计需另外区分左右端点。

查询按左端块号、右端蛇形排序；先扩张再收缩，保证删除位置仍在当前区间。回调 O(1) 时，正块长 B 的时间 O(q log(q+1)+qB+n^2/B+n)，额外空间 O(q)。默认 B=max(1,floor(n/sqrt(q)))，q=0 直接返回。n 和 q 须能用 int 表示。P2709 频次平方和须用 long long，最大 n^2=10^10。P2709 当前代码已 AC（30 个测试点，单点最大 69ms）；运行时间排名未核查。

模板题使用：P2709 莫队 / 小 B 的询问

题意：静态数组值域 1..k，询问闭区间内各值出现次数的平方和。n,m,k≤100000，输入 1-based 闭区间转成 0-based 半开区间；加删一个值由平方差 O(1) 更新，答案用 long long。

所需模板：Mo（第 43 页，5.3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P2709

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    Mo mo(n);
    for (int i = 0; i < m; i++)
    {
        int l, r;
        cin >> l >> r;
        mo.add(l - 1, r);
    }
    vector<int> cnt(k + 1);
    vector<long long> ans(m);
    long long sum = 0;
    auto add = [&](int i)
    {
        sum += 2LL * cnt[a[i]] + 1;
        cnt[a[i]]++;
    };
    auto del = [&](int i)
    {
        cnt[a[i]]--;
        sum -= 2LL * cnt[a[i]] + 1;
    };
    mo.run(add, del, [&](int id)
    {
        ans[id] = sum;
    });
    for (long long x : ans) cout << x << '\n';
}
```

5.3.2 带修改莫队

5.3.2.1 带修改的莫队

ModifiedMo(n,m) 保存离线询问，n>0，m 为修改总数。add(l,r,t) 登记第 t 次修改完成后的 [l,r)，返回从 0 开始的询问编号；0≤l≤r≤n，0≤t≤m，询问数小于 INT_MAX。run(add,del,chg,ans,block=0) 排序并移动三个指针。add(p)/del(p) 按下标更新当前区间统计；chg(i,forward,l,r) 将编号 i 的修改前进或回退，并同步修正当前 [l,r) 内的贡献；ans(id) 记录结果。修改编号从 0 开始，前进第 i 次修改时原时间为 i，回退时原时间为 i+1。回调初始状态必须为原数组、空区间、时间 0；重复 run 前由调用方重置状态。状态不会自动恢复，原询问顺序不变。仅适用于可增删维护的区间统计，需区分左右端操作时另行拆分回调。默认块长约 n^(2/3)，也可显式指定正块长。n、询问数、修改数同阶且回调 O(1) 时总时间 O(n^(5/3))，调度器空间 O(q)。

```
struct ModifiedMo
{
    struct Query
    {
        int l, r, t, id;
    };

    int n, m;
    vector<Query> q;

    ModifiedMo(int n, int m) : n(n), m(m) { assert(n > 0 && m >= 0); }

    int add(int l, int r, int t)
    {
        assert(0 <= l && l <= r && r <= n);
        assert(0 <= t && t <= m);
        int id = q.size();
        q.push_back({l, r, t, id});
        return id;
    }
```

```
    template <class Add, class Del, class Chg, class Ans>
    void run(Add add, Del del, Chg chg, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        if (block == 0) block = max(1, int(pow(n, 2.0 / 3)));
        assert(block > 0);
        auto ord = q;
        auto cmp = [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            int p = a.r / block, q = b.r / block;
            if (p != q) return (x & 1) ? p > q : p < q;
            return ((x ^ p) & 1) ? a.t > b.t : a.t < b.t;
        };
        sort(ord.begin(), ord.end(), cmp);
        int l = 0, r = 0, t = 0;
        for (auto q : ord)
        {
            while (t < q.t)
            {
                chg(t, true, l, r);
                t++;
            }
            while (t > q.t)
            {
                t--;
                chg(t, false, l, r);
            }
            while (l > q.l) add(--l);
            while (r < q.r) add(r++);
            while (l < q.l) del(l++);
            while (r > q.r) del(--r);
            ans(q.id);
        }
    }
};
```

模板题使用：Luogu P1903

题意：单点改色、区间不同颜色数，n,m≤133333，颜色 1..1000000。先读完操作，cur 仅用于记录每次修改前后的颜色，a 保持初始数组。题目闭区间 [L,R] 转为 [L-1,R)，询问时间是此前修改数。chg 必须先删旧贡献、改 a[p]、再加新贡献，回退使用 pre。按登记顺序输出。三维莫队，回调 O(1)，同阶规模时 O(n^(5/3))。

所需模板：ModifiedMo（第 45 页，5.3.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1903

Luogu P1903：使用代码

```
int main()
{
    int n, m;
    scanf("%d%d", &n, &m);
    vector<int> a(n);
    for (int &v : a) scanf("%d", &v);
    vector<int> cur = a;
    vector<array<int, 3>> upd;
    vector<array<int, 3>> qry;
    for (int i = 0; i < m; i++)
    {
        char op;
        int x, y;
        scanf(" %c%d%d", &op, &x, &y);
        x--;
        if (op == 'Q')
            qry.push_back({x, y, int(upd.size())});
        else
        {
            upd.push_back({x, cur[x], y});
            cur[x] = y;
        }
    }
    ModifiedMo mo(n, upd.size());
    for (auto [l, r, t] : qry) mo.add(l, r, t);
    vector<int> cnt(1000001), res(qry.size());
    int tot = 0;
    auto add = [&](int p)
    {
        if (cnt[a[p]]++ == 0) tot++;
    };
    auto del = [&](int p)
    {
        if (--cnt[a[p]] == 0) tot--;
    };
    auto chg = [&](int t, bool fwd, int l, int r)
    {
        auto [p, pre, nxt] = upd[t];
        bool in = l <= p && p < r;
        if (in) del(p);
        a[p] = fwd ? nxt : pre;
        if (in) add(p);
    };
    auto ans = [&](int id)
    {
        res[id] = tot;
    };
    mo.run(add, del, chg, ans);
    for (int v : res) printf("%d\n", v);
}
```

5.3.3 树上莫队

5.3.3.1 树上莫队：点权与边权

代码依赖：HLD（第 83 页，E.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

先对非空连通树 HLD.build(root)，再 TreeMo(tree)；tree 须活到 run 结束且不能修改。点号 1..n，双欧拉序长 2n，n<=INT_MAX/2；DFS 递归。add(u,v,edge=false) 返回 0-based 查询 ID；默认点权，edge=true 把父边权记在子点并排除 LCA。run(add,del,ans,block=0) 的 add/del 参数是顶点，ans 参数是查询 ID；只适合与集合排列无关且可增删的统计。调用前统计为空，结束恢复空；不可在回调内修改树/查询，可重复运行。混用点边查询时回调须有一致的数据解释。默认块长 max(1,2n/sqrt(m))；自定 B>0。m>0 时调度 O(m log(m+1)+mB+n^2/B+n)，查询转换另需 O(m log n)，回调代价另计。空间 O(n+m)，复用 HLD；整数大小另须满足内存限制。

```
struct TreeMo
{
    struct Query
    {
        int l, r, extra, id;
    };

    const HLD &tree;
    vector<int> in, out, euler;
    vector<Query> q;

    TreeMo(const HLD &tree) : tree(tree), in(tree.n + 1), out(tree.n + 1)
    {
        assert(tree.n > 0 && tree.timer == tree.n);
        dfs(tree.rk[1], 0);
    }

    void dfs(int u, int p)
    {
        in[u] = euler.size();
        euler.push_back(u);
        for (int v : tree.g[u])
        {
            if (v == p) continue;
            dfs(v, u);
        }
        out[u] = euler.size();
        euler.push_back(u);
    }

    int add(int u, int v, bool edge = false)
    {
        assert(1 <= u && u <= tree.n && 1 <= v && v <= tree.n);
        if (in[u] > in[v]) swap(u, v);
        int w = tree.lca(u, v);
        int l = w == u ? in[u] : out[u];
        int extra = w == u ? 0 : w;
        if (edge) extra = w == u ? w : 0;
        int id = q.size();
        q.push_back({l, in[v] + 1, extra, id});
        return id;
    }
```

查询调度与状态恢复（接上页同一结构体）：

```
    template <class Add, class Del, class Ans>
    void run(Add add, Del del, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        int n = euler.size();
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            return (x & 1) ? a.r > b.r : a.r < b.r;
        });
        vector<bool> active(tree.n + 1);
        auto toggle = [&](int u)
        {
            if (active[u]) del(u);
            else add(u);
            active[u] = !active[u];
        };
        int l = 0, r = 0;
        for (auto a : ord)
        {
            while (l > a.l) toggle(euler[--l]);
            while (r < a.r) toggle(euler[r++]);
            while (l < a.l) toggle(euler[l++]);
            while (r > a.r) toggle(euler[--r]);
            if (a.extra) toggle(a.extra);
            ans(a.id);
            if (a.extra) toggle(a.extra);
        }
        for (int u = 1; u <= tree.n; u++)
        {
            if (active[u]) del(u);
        }
    }
};
```

应用题补充：SP10707 COT2 - Count on a tree II

题意：静态树路径不同点权数，n<=40000,m<=100000；离散化权值，双欧拉序维护出现次数，额外补 LCA。竞赛应用；边权语义另由核心对拍验证。同一驱动已通过用户模板题 U104310（n,m≤10^5，记录 302633483）；SPOJ 转交记录仍 Waiting。

所需模板：HLD（第 83 页，E.1.1 节）、TreeMo（第 48 页，5.3.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP10707

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n + 1), values;
    for (int u = 1; u <= n; u++)
    {
        cin >> a[u];
        values.push_back(a[u]);
    }
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
    vector<int> color(n + 1), count(values.size());
    for (int u = 1; u <= n; u++)
    {
        color[u] = lower_bound(values.begin(), values.end(), a[u]) - values.begin();
    }
    HLD tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        tree.add(u, v);
    }
    tree.build();
    TreeMo mo(tree);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        mo.add(u, v);
    }
```

继续处理操作（接上页同一 main）：

```
    vector<int> answer(m);
    int distinct = 0;
    auto add = [&](int u)
    {
        if (count[color[u]]++ == 0) distinct++;
    };
    auto del = [&](int u)
    {
        if (--count[color[u]] == 0) distinct--;
    };
    mo.run(add, del, [&](int id)
    {
        answer[id] = distinct;
    });
    for (int x : answer) cout << x << '\n';
}
```

5.3.4 回滚莫队

5.3.4.1 回滚莫队

RollbackMo(n)，n>=0；add(l,r) 登记 0-based 半开区间 [l,r)，允许空区间，返回 0-based 查询 ID。run(add, snapshot, rollback, ans, block=0)：add 接收位置，snapshot 返回当前快照，rollback 恢复该祖先快照，ans 接收查询 ID。开始时统计必须为空；结束恢复初始快照。每次 add 都须可撤销，包含长期右段；不保留失效快照、不修改查询。只支持与插入次序无关的区间统计。默认 B=max(1,n/sqrt(m))；m>0 时调度与 O(1) 增删总计 O(m log(m+1)+mB+n^2/B+n)，空间 O(m) 加回调状态/历史 O(n)。可重复 run，n=0 只允许空查询；无查询不调用回调。

```
struct RollbackMo
{
    struct Query
    {
        int l, r, id;
    };

    int n;
    vector<Query> q;

    RollbackMo(int n) : n(n)
    {
        assert(n >= 0);
    }

    int add(int l, int r)
    {
        assert(0 <= l && l <= r && r <= n);
        int id = q.size();
        q.push_back({l, r, id});
        return id;
    }

    template <class Add, class Snap, class Undo, class Ans>
    void run(Add add, Snap snapshot, Undo rollback, Ans ans, int block = 0) const
    {
        if (q.empty()) return;
        if (block == 0) block = max(1, int(n / sqrt(double(q.size()))));
        assert(block > 0);
        auto ord = q;
        sort(ord.begin(), ord.end(), [&](const Query &a, const Query &b)
        {
            int x = a.l / block, y = b.l / block;
            if (x != y) return x < y;
            return a.r < b.r;
        });
```

查询调度与状态恢复（接上页同一结构体）：

```
        auto empty = snapshot();
        int last = -1, r = 0;
        for (auto a : ord)
        {
            int b = a.l / block;
            int end = a.l + min(block - a.l % block, n - a.l);
            if (b != last)
            {
                rollback(empty);
                last = b;
                r = end;
            }
            if (a.r <= end)
            {
                auto saved = snapshot();
                for (int i = a.l; i < a.r; i++) add(i);
                ans(a.id);
                rollback(saved);
                continue;
            }
            while (r < a.r) add(r++);
            auto saved = snapshot();
            for (int i = end; i > a.l; ) add(--i);
            ans(a.id);
            rollback(saved);
        }
        rollback(empty);
    }
};
```

模板题使用：P5906 【模板】回滚莫队 & 不删除莫队

题意：n,m<=200000，值 1..2e9；对每个 1-based 闭区间输出相同值的最远下标差，无重复输出 0。离散化，保存各值最左/最右位置和全局答案，历史逐次恢复。

所需模板：RollbackMo（第 52 页，5.3.4.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5906

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<int> a(n), values;
    for (int &x : a) cin >> x;
    values = a;
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
    for (int &x : a)
    {
        x = lower_bound(values.begin(), values.end(), x) - values.begin();
    }
    int m;
    cin >> m;
    RollbackMo mo(n);
    for (int i = 0; i < m; i++)
    {
        int l, r;
        cin >> l >> r;
        mo.add(l - 1, r);
    }
```

继续处理操作（接上页同一 main）：

```
    struct Change
    {
        int color, first, last, best;
    };
    vector<Change> history;
    vector<int> first(values.size(), n), last(values.size(), -1), answer(m);
    int best = 0;
    auto add = [&](int i)
    {
        int c = a[i];
        history.push_back({c, first[c], last[c], best});
        first[c] = min(first[c], i);
        last[c] = max(last[c], i);
        best = max(best, last[c] - first[c]);
    };
    auto snapshot = [&]()
    {
        return history.size();
    };
    auto rollback = [&](size_t saved)
    {
        while (history.size() > saved)
        {
            auto h = history.back();
            history.pop_back();
            first[h.color] = h.first;
            last[h.color] = h.last;
            best = h.best;
        }
    };
    mo.run(add, snapshot, rollback, [&](int id)
    {
        answer[id] = best;
    });
    for (int x : answer) cout << x << '\n';
}
```

5.3.5 莫队二次离线

5.3.5.1 莫队二次离线：异或位数配对

xor_hamming_pairs(a,queries,k,bits=14) 按查询输入顺序返回 long long 答案，统计 l≤i<j<r 且 popcount(a[i] xor a[j])=k 的下标对。queries 为 0-based 半开区间，允许空区间、重复查询与空数组；0≤bits≤20，0≤a[i]<2^bits，n、m 和容器下标须可由 int 表示。k 超出 [0,bits] 直接返回全零；相等值来自不同下标才构成一对。莫队移动生成前缀事件，第二次离线按前缀扫描；k=0 显式扣掉自身。令 C=binom(bits,k)，时间 O(2^bits+nC+m log(m+1)+n$\sqrt{m}$+m)，空间 O(2^bits+n+m)，m=0 直接返回。无需维护可逆的在线窗口，也不支持修改。

```
// Queries are 0-based half-open; count unordered pairs of distinct indices.
vector<long long> xor_hamming_pairs(const vector<int> &a,
                                    const vector<pair<int, int>> &queries,
                                    int k,
                                    int bits = 14)
{
    int n = a.size(), m = queries.size();
    vector<long long> answer(m);
    assert(0 <= bits && bits <= 20);
    int universe = 1 << bits;
    for (int x : a) assert(0 <= x && x < universe);
    for (auto [l, r] : queries) assert(0 <= l && l <= r && r <= n);
    if (!m || k < 0 || k > bits) return answer;
    vector<int> masks;
    for (int x = 0; x < universe; x++)
        if (popcount(unsigned(x)) == k) masks.push_back(x);
    vector<int> count(universe);
    vector<long long> prefix(n + 1);
    for (int i = 0; i < n; i++)
    {
        prefix[i + 1] = prefix[i] + count[a[i]];
        for (int mask : masks) count[a[i] ^ mask]++;
    }
    int block = max(1, int(n / sqrt(m)));
    vector<int> order(m);
    iota(order.begin(), order.end(), 0);
    sort(order.begin(),
         order.end(),
         [&](int x, int y)
         {
             int bx = queries[x].first / block, by = queries[y].first / block;
             if (bx != by) return bx < by;
             return bx & 1 ? queries[x].second > queries[y].second
                           : queries[x].second < queries[y].second;
         });

    struct Event
    {
        int id, l, r, sign;
    };

    vector<vector<Event>> events(n + 1);
```

莫队端点移动与前缀事件（接上页同一函数）：

```
    int l = 0, r = 0;
    for (int id : order)
    {
        auto [ql, qr] = queries[id];
        if (r < qr)
        {
            events[l].push_back({id, r, qr, -1});
            answer[id] += prefix[qr] - prefix[r];
            r = qr;
        }
        if (l > ql)
        {
            events[r].push_back({id, ql, l, 1});
            answer[id] -= prefix[l] - prefix[ql];
            l = ql;
        }
        if (r > qr)
        {
            events[l].push_back({id, qr, r, 1});
            answer[id] -= prefix[r] - prefix[qr];
            r = qr;
        }
        if (l < ql)
        {
            events[r].push_back({id, l, ql, -1});
            answer[id] += prefix[ql] - prefix[l];
            l = ql;
        }
    }
```

第二次离线与差分还原（接上页同一函数）：

```
    fill(count.begin(), count.end(), 0);
    for (int p = 0; p <= n; p++)
    {
        if (p)
            for (int mask : masks) count[a[p - 1] ^ mask]++;
        for (auto e : events[p])
            for (int i = e.l; i < e.r; i++)
            {
                int value = count[a[i]] - (k == 0 && i < p);
                answer[e.id] += 1LL * e.sign * value;
            }
    }
    long long total = 0;
    for (int id : order)
    {
        total += answer[id];
        answer[id] = total;
    }
    return answer;
}
```

模板题使用：Luogu P4887

题意：统计每个区间内异或值恰有 k 个 1 的无序下标对。输入 1-based 闭区间 [l,r]，只将 l 减一转为 0-based 半开区间；k 可以大于 14，此时全零。k=0 统计相等值的不同下标，不计自身。

所需模板：xor_hamming_pairs（第 56 页，5.3.5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4887

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    vector<pair<int, int>> queries(m);
    for (auto &[l, r] : queries)
    {
        cin >> l >> r;
        l--;
    }
    for (long long x : xor_hamming_pairs(a, queries, k)) cout << x << '\n';
}
```

第六章　配套知识与验证范围

6.1　带修莫队：时间轴与区间状态

先离线读完操作，记录每个单点修改的下标、修改前值和修改后值。询问的时间 t 表示前 t 次修改已执行；输入中的询问本身不推进时间。同一位置的第二次修改，其旧值必须来自第一次修改后的数组，不能始终使用初始值。需要离散化时，应将全部初值与修改后值一同纳入。

调度器保持三个指针 (l, r, t)，回调统计恰好对应当前版本的半开区间 $[l, r)$。前进修改 i 时写入 after，回退时写入 before。若修改点在当前区间内，必须先删除旧贡献、改值、再加入新贡献；区间外只改数组。随后移动区间端点，最后按原询问编号写答案。扩展区间先于收缩，支持空区间和互不相交区间。整个 run 结束后状态停留在排序后的最后一条询问；再次调用前须重置原数组和统计量。

左端点分块、右端点分块，再按时间排序，相邻块采用蛇形顺序。时间关键字是真实修改次数，不能把询问编号当作时间维度。令块长为 B，询问数为 Q，修改数为 U，则常数时间回调下的移动量上界为

$$O\left(QB + \frac{n^2}{B} + U\min\left(Q, (\lfloor n/B \rfloor + 1)^2\right)\right).$$

另有 $O(Q \log Q)$ 排序与 $O(Q)$ 调度空间。三者同阶时取 $B \approx n^{2/3}$，得到 $O(n^{5/3})$；规模悬殊时可手动调整块长。回调需要区分左右端增删时应拆分四个方向，当前简版适合不同数、频次矩等统计。

P1903 数颜色 驱动以颜色频次数组维护不同数，真实操作顺序建立修改前后值，随后交给调度器。本地快照测试逐项检查每个时间、每个区间及修改回退状态，另以顺序 set 查询核对完整驱动。最大规模验证 133333 个元素和操作，并覆盖仅修改、仅询问与重复修改同一位置。最终简短命名版本在 P1903 记录 297653743 通过全部 13 个测试，最慢单点 546 ms、峰值内存 14.65 MB。此记录使用默认块长与不同颜色数统计；自定义块长、空区间及任意登记时间由本地测试覆盖。最快提交榜的排名及总数尚未核验。

6.2 莫队二次离线：前缀事件与自身扣除

对区间内满足 $H(i,j) = [\text{popcount}(a_i \text{ xor } a_j) = k]$ 的无序下标对，使用 `xor_hamming_pairs`。令 $t(i) = \sum_{j<i} H(i,j)$，$P(x) = \sum_{i<x} t(i)$，以及 $T(p,i) = \sum_{j<p} H(i,j) - [k = 0 \wedge i < p]$。右端扩展时贡献为 $t(i) - T(L,i)$，左端扩展为 $T(R,i) - t(i)$；删除取相反数。整段端点移动的 t 和由 P 的区间差求出，剩余部分存成按前缀长度分桶的事件。第二遍顺扫只做插入，每次枚举异或后有 k 个置位的掩码更新计数；事件中的每个下标只读一次计数，并在 $k = 0$ 时扣自身。最后按莫队顺序累加窗口答案差，再按原编号返回。

区间统一为 0-based 半开区间，先扩右、扩左，再缩右、缩左，保持窗口合法。允许空区间；没有查询时直接返回。位宽默认为 14，若 k 超出位宽则答案全零。P4887 的 $k < 2^{14}$ 并不代表 $k \le 14$。令 $C = \binom{b}{k}$，块长约为 $n/\sqrt{m}$，总时间 $O(2^b + nC + m\log(m+1) + n\sqrt{m} + m)$，空间 $O(2^b + n + m)$。端点移动不再乘枚举掩码数 C。答案与前缀和使用 long long；十万个相等数的配对数超过 int32。本地全枚举、随机逐对参考及十万规模的 $k = 0, 7$ 分支通过；线上验证待补。

第一部分

跨类依赖

附录 A　数论

A.1　模算术简介

A.1.1　静态模整数

固定 int 模数 1≤mod≤INT_MAX，可为合数。构造接受 signed64 并正规化，v 保持 0..mod-1；不要手动破坏正规化。加减乘 O(1)，pow 非负 signed64 指数、时间 O(log(e+1))。try_inv 用欧几里得求逆：gcd(v,mod)=1 时返回 optional 值，否则 nullopt；mod=1 返回代表 0。inv、/、/= 要求分母可逆；断言开启时诊断违反前提，关闭断言不承诺拒绝非法除法。逆元 O(log(mod+1))、额外空间 O(1)，只用 long long，不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用，支持自赋值与链式运算，可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
template <int mod> struct ModInt
{
    static_assert(mod >= 1);
    int v;

    ModInt(long long x = 0) : v((x % mod + mod) % mod) {}

    ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }

    ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }

    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }

    ModInt pow(long long e) const
    {
        assert(e >= 0);
        ModInt a = *this, r = 1;
        for (; e; e >>= 1, a = a * a)
            if (e & 1) r = r * a;
        return r;
    }
```

求逆与除法（接上页同一结构体）：

```
    optional<ModInt> try_inv() const
    {
        long long a = v, b = mod, x = 1, y = 0;
        while (b)
        {
            long long q = a / b;
            a -= q * b;
            swap(a, b);
            x -= q * y;
            swap(x, y);
        }
        if (a != 1) return nullopt;
        return ModInt(x);
    }

    ModInt inv() const
    {
        auto r = try_inv();
        assert(r);
        return *r;
    } // requires an invertible residue

    ModInt operator/(ModInt b) const { return *this * b.inv(); }

    ModInt &operator+=(ModInt b) { return *this = *this + b; }

    ModInt &operator-=(ModInt b) { return *this = *this - b; }

    ModInt &operator*=(ModInt b) { return *this = *this * b; }

    ModInt &operator/=(ModInt b) { return *this = *this / b; }
};
```

联合使用示例见第 67 页（Luogu P1313 计算系数），其中直接调用本模板。

A.2 卢卡斯定理

A.2.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p，choose(n,k) 返回 C(n,k) 模 p；n、k 为完整 unsigned long long 范围，k>n 返回 0，C(0,0)=1。重新赋值 Lucas(p) 可更换模数并重新预处理；不检测素性，要求调用者保证 p 为素数且 p≥2。预处理时间 O(p+log p)，动态版空间 O(p)；单次查询 O(1+log_p(n+1))。适合小素数模数，通常 $p \le 10^6$；大素数不能因查询下标小就忽略这里整段 O(p) 的预处理。fac、ifac 下标为 0..p-1，模数和表只读；中间乘法用 long long。合数模数请使用 exLucas。

```
struct Lucas
{
    int mod;
    vector<int> fac, ifac;

    Lucas(int p)
    {
        assert(p >= 2);
        mod = p;
        fac.resize(p);
        ifac.resize(p);
        fac[0] = 1;
        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
        long long a = fac[p - 1], r = 1;
        for (int e = p - 2; e; e >>= 1)
        {
            if (e & 1) r = r * a % p;
            a = a * a % p;
        }
        ifac[p - 1] = r;
        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
    }
```

```
    int choose(unsigned long long n, unsigned long long k) const
    {
        if (k > n) return 0;
        long long answer = 1;
        while (n || k)
        {
            int a = n % mod, b = k % mod;
            if (b > a) return 0;
            answer = answer * fac[a] % mod;
            answer = answer * ifac[b] % mod;
            answer = answer * ifac[a - b] % mod;
            n /= mod;
            k /= mod;
        }
        return answer;
    }
};
```

模板题使用：Luogu P3807

题意：多组查询 C(n+m,n) 模 p，题目保证 p 为素数且不超过 10^5。每组构造 Lucas(p)，choose 的参数为 n+m、n；此题范围内 n+m 不会溢出。

所需模板：Lucas（第 64 页，A.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3807

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--)
    {
        unsigned long long n, m;
        int p;
        cin >> n >> m >> p;
        Lucas solver(p);
        cout << solver.choose(n + m, n) << '\n';
    }
    return 0;
}
```

附录 B　组合数学

B.1　排列组合

B.1.1　阶乘组合数

代码依赖：ModInt（第 62 页，A.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

Binomial<prime>(n=0) 预处理 0..n，要求 prime 为素数且 0≤n<prime。init(n) 只扩大当前上限，较小或相同的 n 不改变表；choose(n,k) 返回 C(n,k)，permute(n,k) 返回 n!/(n-k)!。查询前显式预处理到 n，查询本身不扩表。n<0、k<0 或 k>n 返回 0，C(0,0)=P(0,0)=1。fac、ifac 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询 O(1)；从 old 扩到 n 新增 O(n-old+log prime) 次域运算，重分配还可能复制旧表。首次预处理 O(n+log prime)，空间 O(n)。超过模数不能使用阶乘除法：小素数用 Lucas，小合数用 exLucas。

```
template <int mod> struct Binomial
{
    using Z = ModInt<mod>;
    vector<Z> fac{Z(1)}, ifac{Z(1)};

    Binomial(int n = 0) { init(n); }

    void init(int n)
    {
        assert(0 <= n && n < mod);
        int old = (int)fac.size() - 1;
        if (n <= old) return;
        fac.resize(n + 1);
        ifac.resize(n + 1);
        for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
        ifac[n] = fac[n].inv();
        for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
    }

    Z choose(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[k] * ifac[n - k];
    }

    Z permute(int n, int k) const
    {
        if (k < 0 || k > n) return 0;
        assert(n < (int)fac.size());
        return fac[n] * ifac[n - k];
    }
};
```

应用题补充：Luogu P1313 计算系数

题意：求 (by+ax) 的 k 次方中指定项的系数，模 10007。x、y 的指数分别为 n、m，均非负且 n+m=k。k 至多 1000，a、b 均为不超过 1000000 的非负数。组合 C(k,n) 与两个幂即可。模数为素数且 k 小于模数，满足阶乘逆元前提；零次幂按空乘积为 1。

所需模板：ModInt（第 62 页，A.1.1 节）、Binomial（第 66 页，B.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1313

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int a, b, k, n, m;
    cin >> a >> b >> k >> n >> m;
    Binomial<10007> c(k);
    auto x = ModInt<10007>(a).pow(n);
    auto y = ModInt<10007>(b).pow(m);
    cout << (c.choose(k, n) * x * y).v << '\n';
}
```

附录 C　树状数组

C.1　树状数组第 k 小与组合依赖

保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
template <class T = long long> struct Fenwick
{
    int n;
    vector<T> a;

    Fenwick(int n = 0) : n(n), a(n + 1) {}

    void add(int x, T v)
    {
        assert(x >= 1);
        for (; x <= n; x += x & -x) a[x] += v;
    }

    T sum(int x) const
    {
        T ans = 0;
        for (; x; x -= x & -x) ans += a[x];
        return ans;
    }

    T query(int l, int r) const { return sum(r) - sum(l - 1); }

    // Nonnegative frequencies; returns n+1 if k exceeds total.
    int kth(T k) const
    {
        assert(k > 0);
        int x = 0, b = 1;
        while (b <= n / 2) b *= 2;
        for (; b; b >>= 1)
            if (x + b <= n && a[x + b] < k)
            {
                k -= a[x + b];
                x += b;
            }
        return x + 1;
    }
};
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based，kth 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。$n \le 100000$，$|x| \le 10000000$；总复杂度 $O(n \log n)$，空间 $O(n)$。

所需模板：Fenwick（第 68 页，C.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3369

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<pair<int, int>> q(n);
    vector<int> a;
    for (auto &[op, x] : q)
    {
        cin >> op >> x;
        if (op != 4) a.push_back(x);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    Fenwick<int> f(a.size());
    for (auto [op, x] : q)
    {
        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
        if (op == 1)
            f.add(p + 1, 1);
        else if (op == 2)
        {
            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
        }
        else if (op == 3)
            cout << f.sum(p) + 1 << '\n';
        else if (op == 4)
            cout << a[f.kth(x) - 1] << '\n';
        else if (op == 5)
            cout << a[f.kth(f.sum(p)) - 1] << '\n';
        else
        {
            p = upper_bound(a.begin(), a.end(), x) - a.begin();
            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
        }
    }
}
```

附录 D　线段树

D.1　线段树基础

D.1.1　通用懒标记线段树与边界搜索

模板参数依次为 S、F、op、e、mapping、composition、id；以数组 a 构造，0-based 半开区间；n 构造将 n 个元素置为 e。0≤n≤2^28-1，受内存限制。op 满足结合律、e 为双侧单位元；单位标记不改变节点，mapping 必须保持 op 的合并结果；composition(f,g) 表示先 g 后 f，须满足作用复合一致性，id 为标记单位元。无需比较标记是否等于 id，每次下传都应用标记。set/get 单点，apply(l,r,f) 区间作用，prod(l,r) 保序合并，all() 读取根；空区间不更新且查询返回 e。区间长度若参与修改，必须存入 S，不能从 e 推断长度：区间和节点为 {sum,len}，真实叶 len=1，e={0,0}。max_right/min_left 的谓词须满足 ok(e())、无副作用、扩展时真变假后不再变真；两者查询会下传标记，非 const。递归深度 O(log n)，构造 O(n)、操作/搜索 O(log n)、all O(1)，空间 O(n)，假设运算和类型复制 O(1)。d/tag 及其他递归辅助方法仅供内部使用。

```
template <class S, class F, auto op, auto e, auto mapping, auto composition, auto id>
struct lazy_segtree
{
    int n;
    vector<S> d;
    vector<F> tag;

    lazy_segtree(int n = 0) : lazy_segtree(vector<S>(n, e())) {}

    lazy_segtree(const vector<S> &a)
        : n(a.size()), d(4 * n + 4, e()), tag(4 * n + 4, id())
    {
        if (n) build(1, 0, n, a);
    }

    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void build(int p, int l, int r, const vector<S> &a)
    {
        if (r - l == 1)
        {
            d[p] = a[l];
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m, r, a);
        pull(p);
    }
```

```
    void act(int p, F f)
    {
        d[p] = mapping(f, d[p]);
        tag[p] = composition(f, tag[p]);
    }

    void push(int p)
    {
        act(p * 2, tag[p]);
        act(p * 2 + 1, tag[p]);
        tag[p] = id();
    }

    void set(int x, S v)
    {
        assert(0 <= x && x < n);
        put(1, 0, n, x, v);
    }

    void put(int p, int l, int r, int x, S v)
    {
        if (r - l == 1)
        {
            d[p] = v;
            tag[p] = id();
            return;
        }
        push(p);
        int m = (l + r) / 2;
        if (x < m)
            put(p * 2, l, m, x, v);
        else
            put(p * 2 + 1, m, r, x, v);
        pull(p);
    }

    void apply(int l, int r, F f)
    {
        assert(0 <= l && l <= r && r <= n);
        if (l < r) update(1, 0, n, l, r, f);
    }

    void update(int p, int l, int r, int a, int b, F f)
    {
        if (a <= l && r <= b)
        {
            act(p, f);
            return;
        }
        push(p);
        int m = (l + r) / 2;
        if (a < m) update(p * 2, l, m, a, b, f);
        if (m < b) update(p * 2 + 1, m, r, a, b, f);
        pull(p);
    }
```

```
87      S all() const { return d[1]; }
88
89      S get(int x)
90      {
91          assert(0 <= x && x < n);
92          return prod(x, x + 1);
93      }
94
95      S prod(int l, int r)
96      {
97          assert(0 <= l && l <= r && r <= n);
98          return l == r ? e() : query(1, 0, n, l, r);
99      }
100
101     S query(int p, int l, int r, int a, int b)
102     {
103         if (a <= l && r <= b) return d[p];
104         push(p);
105         int m = (l + r) / 2;
106         if (b <= m) return query(p * 2, l, m, a, b);
107         if (m <= a) return query(p * 2 + 1, m, r, a, b);
108         return op(query(p * 2, l, m, a, b), query(p * 2 + 1, m, r, a, b));
109     }
110
111     template <class G> int max_right(int l, G ok)
112     {
113         assert(0 <= l && l <= n && ok(e()));
114         S s = e();
115         return l == n ? n : right(1, 0, n, l, s, ok);
116     }
117
118     template <class G> int right(int p, int l, int r, int a, S &s, G &ok)
119     {
120         if (r <= a) return n;
121         if (a <= l && ok(op(s, d[p])))
122         {
123             s = op(s, d[p]);
124             return n;
125         }
126         if (r - l == 1) return l;
127         push(p);
128         int m = (l + r) / 2;
129         int x = right(p * 2, l, m, a, s, ok);
130         return x != n ? x : right(p * 2 + 1, m, r, a, s, ok);
131     }
```

```
    template <class G> int min_left(int r, G ok)
    {
        assert(0 <= r && r <= n && ok(e()));
        S s = e();
        return r == 0 ? 0 : left(1, 0, n, r, s, ok);
    }

    template <class G> int left(int p, int l, int r, int b, S &s, G &ok)
    {
        if (b <= l) return 0;
        if (r <= b && ok(op(d[p], s)))
        {
            s = op(d[p], s);
            return 0;
        }
        if (r - l == 1) return r;
        push(p);
        int m = (l + r) / 2;
        int x = left(p * 2 + 1, m, r, b, s, ok);
        return x != 0 ? x : left(p * 2, l, m, b, s, ok);
    }
};
```

模板题使用：Library Checker range_affine_range_sum

题意：对数组区间施加 x→b*x+c，并查询区间和。S 保存和与长度；空单位元长度 0，单个元素长度 1。composition(f,g) 表示先 g 后 f，mapping 按长度累加常数项。区间为 0-based 半开，模 998244353。

所需模板：lazy_segtree（第 70 页，D.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/range_affine_range_sum

```
using S = pair<long long, int>;
using F = pair<long long, long long>;
const int mod = 998244353;

S op(S a, S b)
{
    return {(a.first + b.first) % mod, a.second + b.second};
}

S e()
{
    return {0, 0};
}

S mapping(F f, S a)
{
    return {(f.first * a.first + f.second * a.second) % mod, a.second};
}

F composition(F f, F g)
{
    return {f.first * g.first % mod, (f.first * g.second + f.second) % mod};
}

F id()
{
    return {1, 0};
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, len] : a)
    {
        cin >> x;
        len = 1;
    }
    lazy_segtree<S, F, op, e, mapping, composition, id> s(a);
    while (q--)
    {
        int t, l, r;
        cin >> t >> l >> r;
        if (t == 0)
        {
            long long b, c;
            cin >> b >> c;
            s.apply(l, r, {b, c});
        }
        else
            cout << s.prod(l, r).first << '\n';
    }
}
```

D.1.2 通用线段树与左右边界搜索

segtree<S,op,e>(a) 从 0-based vector 构建，或 segtree(n) 初始化为单位元。0≤n≤2^29，实际受内存限制。op(S,S) 须满足结合律，e() 是双侧单位元；无需交换律。set(p,x)、get(p) 操作单点，prod(l,r) 返回半开区间 [l,r) 从左到右的合并，空区间返回 e；all() 返回全区间结果。max_right(l,ok) 返回最大的 r 使 ok(prod(l,r)) 为真；min_left(r,ok) 返回最小的 l。要求 ok(e()) 为真，ok 无副作用，并随对应方向扩展从真变假后不再变真；例如非负和不超过阈值，不能对任意负数和这样搜索。l=n、r=0 支持空端点。构建 O(n)，单点/区间/边界搜索 O(log n)，all O(1)，空间 O(n)，假设 op、e、ok 及 S 复制为 O(1)。d、sz、pull 为内部实现，不直接修改。函数复合时 op(f,g) 表示 g(f(x))，对应先执行左区间再执行右区间；min_left 不能照抄 max_right 的合并顺序。

```
template <class S, auto op, auto e> struct segtree
{
    int n, sz = 1;
    vector<S> d;

    segtree(int n = 0) : segtree(vector<S>(n, e())) {}

    segtree(const vector<S> &a) : n(a.size())
    {
        while (sz < n) sz <<= 1;
        d.assign(2 * sz, e());
        copy(a.begin(), a.end(), d.begin() + sz);
        for (int i = sz - 1; i; i--) pull(i);
    }

```

```
    void pull(int p) { d[p] = op(d[p * 2], d[p * 2 + 1]); }

    void set(int p, S x)
    {
        assert(0 <= p && p < n);
        d[p += sz] = x;
        while (p >>= 1) pull(p);
    }

    S get(int p) const
    {
        assert(0 <= p && p < n);
        return d[p + sz];
    }

    S all() const { return d[1]; }
```

```
    S prod(int l, int r) const
    {
        assert(0 <= l && l <= r && r <= n);
        S a = e(), b = e();
        for (l += sz, r += sz; l < r; l >>= 1, r >>= 1)
        {
            if (l & 1) a = op(a, d[l++]);
            if (r & 1) b = op(d[--r], b);
        }
        return op(a, b);
    }

    template <class F> int max_right(int l, F ok) const
    {
        assert(0 <= l && l <= n && ok(e()));
        if (l == n) return n;
        S a = e();
        l += sz;
        do
        {
            while (!(l & 1)) l >>= 1;
            if (!ok(op(a, d[l])))
            {
                while (l < sz)
                {
                    l *= 2;
                    if (ok(op(a, d[l]))) a = op(a, d[l++]);
                }
                return l - sz;
            }
            a = op(a, d[l++]);
        } while ((l & -l) != l);
        return n;
    }
```

```
    template <class F> int min_left(int r, F ok) const
    {
        assert(0 <= r && r <= n && ok(e()));
        if (r == 0) return 0;
        S a = e();
        r += sz;
        do
        {
            --r;
            while (r > 1 && (r & 1)) r >>= 1;
            if (!ok(op(d[r], a)))
            {
                while (r < sz)
                {
                    r = r * 2 + 1;
                    if (ok(op(d[r], a))) a = op(d[r--], a);
                }
                return r + 1 - sz;
            }
            a = op(d[r], a);
        } while ((r & -r) != r);
        return 0;
    }
};
```

模板题使用：Library Checker point_set_range_composite

题意：单点修改一次函数，查询区间内从左到右依次作用于 x 的结果。op(f,g) 表示先 f 后 g，单位元为 x；prod(l,r) 使用 0-based 半开区间，空区间返回单位元。配置函数须和 main 一起抄写。

所需模板：segtree（第 75 页，D.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/point_set_range_composite

```
using S = pair<long long, long long>;
const int mod = 998244353;

S op(S f, S g)
{
    return {f.first * g.first % mod, (f.second * g.first + g.second) % mod};
}

S e()
{
    return {1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n);
    for (auto &[x, y] : a) cin >> x >> y;
    segtree<S, op, e> s(a);
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(p, {c, d});
        }
        else
        {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            auto [c, d] = s.prod(l, r);
            cout << (c * x + d) % mod << '\n';
        }
    }
}
```

D.1.3 线段树：区间乘、加、赋值与模意义求和

AffineSegTree(a,mod) 传入非空 0-based 初值数组；查询与修改采用 1-based 非空闭区间。update(l,r,mul,add) 将区间元素 x 改为 mul*x+add；乘法用 (mul,0)，加法用 (1,add)，赋值用 (0,value)。query(l,r) 返回区间和。1≤mod≤INT_MAX，模数无需为素数，初值和修改参数允许 long long 全范围，入口统一归一化。旧标记 (m,b) 后应用新标记 (a,c)，复合为 (m*a,b*a+c)，乘法取模后再相加。建树 O(n)，单次 O(log n)，空间 O(n)。内部 build、apply 等函数不作为公共接口；动态版重建新对象。

```
struct AffineSegTree
{
    using ll = long long;

    struct Node
    {
        ll sum = 0, mul = 1, add = 0;
    };

    int n, mod;
    vector<Node> t;

    AffineSegTree(const vector<ll> &a, int mod) : n(a.size()), mod(mod), t(4 * n + 4)
    {
        assert(n > 0 && mod > 0);
        build(1, 1, n, a);
    }

    ll norm(ll x) const
    {
        x %= mod;
        if (x < 0) x += mod;
        return x;
    }

    void pull(int p) { t[p].sum = (t[p * 2].sum + t[p * 2 + 1].sum) % mod; }

    void build(int p, int l, int r, const vector<ll> &a)
    {
        t[p] = Node{};
        if (l == r)
        {
            t[p].sum = norm(a[l - 1]);
            return;
        }
        int m = (l + r) / 2;
        build(p * 2, l, m, a);
        build(p * 2 + 1, m + 1, r, a);
        pull(p);
    }

    // New transform follows the old pending transform: x -> x*a+b.
    void apply(int p, int len, ll a, ll b)
    {
        t[p].sum = (t[p].sum * a % mod + len * b % mod) % mod;
        t[p].mul = t[p].mul * a % mod;
        t[p].add = (t[p].add * a % mod + b) % mod;
    }
```

```
49
50      void push(int p, int l, int r)
51      {
52          if (t[p].mul == 1 && t[p].add == 0) return;
53          int m = (l + r) / 2;
54          apply(p * 2, m - l + 1, t[p].mul, t[p].add);
55          apply(p * 2 + 1, r - m, t[p].mul, t[p].add);
56          t[p].mul = 1;
57          t[p].add = 0;
58      }
59
60      void update(int l, int r, ll a, ll b)
61      {
62          assert(1 <= l && l <= r && r <= n);
63          update(1, 1, n, l, r, norm(a), norm(b));
64      }
65
66      void update(int p, int l, int r, int ql, int qr, ll a, ll b)
67      {
68          if (ql <= l && r <= qr)
69          {
70              apply(p, r - l + 1, a, b);
71              return;
72          }
73          push(p, l, r);
74          int m = (l + r) / 2;
75          if (ql <= m) update(p * 2, l, m, ql, qr, a, b);
76          if (qr > m) update(p * 2 + 1, m + 1, r, ql, qr, a, b);
77          pull(p);
78      }
79
80      ll query(int l, int r)
81      {
82          assert(1 <= l && l <= r && r <= n);
83          return query(1, 1, n, l, r);
84      }
85
86      ll query(int p, int l, int r, int ql, int qr)
87      {
88          if (ql <= l && r <= qr) return t[p].sum;
89          push(p, l, r);
90          int m = (l + r) / 2;
91          ll answer = 0;
92          if (ql <= m) answer += query(p * 2, l, m, ql, qr);
93          if (qr > m) answer += query(p * 2 + 1, m + 1, r, ql, qr);
94          return answer % mod;
95      }
96  };
```

模板题使用：Luogu P3373

题意：对数组执行区间乘、区间加和区间求和，结果对给定模数取模。区间为 1-based 闭区间；update 的两个参数表示先乘再加。

所需模板：AffineSegTree（第 80 页，D.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3373

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, q, mod;
    std::cin >> n >> q >> mod;
    std::vector<long long> a(n);
    for (auto &x : a) std::cin >> x;
    AffineSegTree tree(a, mod);
    while (q--)
    {
        int op, l, r;
        std::cin >> op >> l >> r;
        if (op == 3)
            std::cout << tree.query(l, r) << '\n';
        else
        {
            long long k;
            std::cin >> k;
            if (op == 1)
                tree.update(l, r, k, 0);
            else
                tree.update(l, r, 1, k);
        }
    }
}
```

附录 E　树上问题

E.1　树链剖分

E.1.1　重链剖分与 LCA

add(u,v) 建无向树；build(root) 后读取 fa、dep、siz、son、top、dfn、rk。子树对应 [dfn[u],dfn[u]+siz[u]-1]。lca(u,v) 为 O(log n)，path(u,v,work,edge) 分解路径；work(l,r) 仅适用于可交换聚合，边权模式排除 LCA 位置。输入必须是连通无环树，建树 O(n)，两遍递归 DFS；支持改变根后重复 build，dfn 会重新编号。path_ordered(u, v, work, edge) 的 edge 默认为 false；按真实 u 到 v 顺序回调 work(l, r, rev)：1-based 闭区间 l≤r，rev=true 按 r 到 l 访问，否则按 l 到 r。支持非交换聚合，O(log n) 段与额外空间；结合线段树通常每次查询 $O(\log^2 n)$。edge=true 仅删除 LCA 对应位置，仍须自行处理边算子的方向；不自动求逆。u=v 时点模式一次单点回调，边模式无回调；改变根后须重建依赖 dfn 的结构。

```
struct HLD
{
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> fa, dep, siz, son, top, dfn, rk;

    HLD(int n)
        : n(n),
          g(n + 1),
          fa(n + 1),
          dep(n + 1),
          siz(n + 1),
          son(n + 1),
          top(n + 1),
          dfn(n + 1),
          rk(n + 1)
    {
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p)
    {
        fa[u] = p;
        siz[u] = 1;
        son[u] = 0;
```

```
        for (int v : g[u])
        {
            if (v == p) continue;
            dep[v] = dep[u] + 1;
            dfs1(v, u);
            siz[u] += siz[v];
            if (!son[u] || siz[v] > siz[son[u]]) son[u] = v;
        }
    }
```

重编号、LCA 与可交换路径（接上页同一结构体）：

```
    void dfs2(int u, int t)
    {
        top[u] = t;
        dfn[u] = ++timer;
        rk[timer] = u;
        if (son[u]) dfs2(son[u], t);
        for (int v : g[u])
        {
            if (v != fa[u] && v != son[u]) dfs2(v, v);
        }
    }

    // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        timer = 0;
        dep[root] = 0;
        dfs1(root, root);
        dfs2(root, root);
        assert(timer == n);
    }

    int lca(int u, int v) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            u = fa[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }

    // Commutative operations only; edge=true excludes the LCA's position.
    template <class F> void path(int u, int v, F work, bool edge = false) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            work(dfn[top[u]], dfn[u]);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v]);
    }
```

按真实方向分解路径（接上页同一结构体）：

```
    // Emit u-to-v order; reverse=true traverses the closed interval from r to l.
    template <class F> void path_ordered(int u, int v, F work, bool edge = false) const
    {
        vector<pair<int, int>> down;
        while (top[u] != top[v])
        {
            if (dep[top[u]] >= dep[top[v]])
            {
                work(dfn[top[u]], dfn[u], true);
                u = fa[top[u]];
            }
            else
            {
                down.emplace_back(dfn[top[v]], dfn[v]);
                v = fa[top[v]];
            }
        }
        if (dep[u] >= dep[v])
        {
            if (dfn[v] + edge <= dfn[u]) work(dfn[v] + edge, dfn[u], true);
        }
        else if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v], false);
        for (auto it = down.rbegin(); it != down.rend(); ++it)
            work(it->first, it->second, false);
    }
};
```

模板题使用：Luogu P3384

题意：给定有根点权树，支持路径加、路径和、子树加、子树和，答案模 mod。先按 dfn 重排初值再构造线段树；path 回调给出 1-based 闭区间，子树区间为 [dfn[u],dfn[u]+siz[u]-1]。

所需模板：HLD（第 83 页，E.1.1 节）、AffineSegTree（第 80 页，D.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3384

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, root, mod;
    cin >> n >> m >> root >> mod;
    vector<long long> values(n + 1);
    for (int u = 1; u <= n; u++) cin >> values[u];
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        h.add(u, v);
    }
    h.build(root);
    vector<long long> ordered(n);
    for (int u = 1; u <= n; u++) ordered[h.dfn[u] - 1] = values[u];
    AffineSegTree seg(ordered, mod);
    while (m--)
    {
        int op, u;
        cin >> op >> u;
        if (op == 1)
        {
            int v;
            long long x;
            cin >> v >> x;
            h.path(u, v, [&](int l, int r) { seg.update(l, r, 1, x); });
        }
        else if (op == 2)
        {
            int v;
            cin >> v;
            long long ans = 0;
            h.path(u, v, [&](int l, int r) { ans = (ans + seg.query(l, r)) % mod; });
            cout << ans << '\n';
        }
        else if (op == 3)
        {
            long long x;
            cin >> x;
            seg.update(h.dfn[u], h.dfn[u] + h.siz[u] - 1, 1, x);
        }
        else
            cout << seg.query(h.dfn[u], h.dfn[u] + h.siz[u] - 1) << '\n';
    }
}
```

模板题使用：Library Checker Vertex Set Path Composite

题意：静态无向树，1≤N,Q≤200000。点函数 f(x)=a*x+b，对 998244353 取模；题面 1≤a,c<mod，0≤b,d,x<mod。0 p c d 改单点函数，1 u v x 按 u 到 v（含两端点）依次代入。题目点号 0-based，传 HLD 时加 1；HLD 闭区间 [l,r] 对应 segtree 半开 [l−1,r)。S 存正向与反向复合，不能把组合顺序反过来或用逆函数代替反向序列。点改 O(log N)，路径 O($\log^2$ N)，空间 O(N)；保留递归 DFS，最大链须足够大栈。

所需模板：HLD（第 83 页，E.1.1 节）、segtree（第 75 页，D.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/vertex_set_path_composite

```
using S = array<long long, 4>;
const int mod = 998244353;

S op(S a, S b)
{
    return {b[0] * a[0] % mod,
            (b[0] * a[1] + b[1]) % mod,
            a[2] * b[2] % mod,
            (a[2] * b[3] + a[3]) % mod};
}

S e()
{
    return {1, 0, 1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n), val(n);
    for (auto &x : a)
    {
        cin >> x[0] >> x[1];
        x[2] = x[0];
        x[3] = x[1];
    }
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        h.add(u + 1, v + 1);
    }
    h.build();
    for (int u = 1; u <= n; u++) val[h.dfn[u] - 1] = a[u - 1];
    segtree<S, op, e> s(val);
```

继续处理操作（接上页同一 main）：

```
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(h.dfn[p + 1] - 1, {c, d, c, d});
        }
        else
        {
            int u, v;
            long long x;
            cin >> u >> v >> x;
            h.path_ordered(u + 1, v + 1, [&](int l, int r, bool rev)
            {
                S z = s.prod(l - 1, r);
                int k = rev ? 2 : 0;
                x = (z[k] * x + z[k + 1]) % mod;
            });
            cout << x << '\n';
        }
    }
}
```

接口用法演示：路径点权增减与点查（kuangbin 3.3.1 源码模型）

演示约定：读到 EOF：n,m,q、n 个初值、m 条树边，要求 m=n−1。I u v x/D u v x 给路径所有点加/减 x，Q u 查点值。1-based，初值保留在 a，Fenwick 仅记录增量差分；HLD 闭区间 [l,r] 在 l 加 x、r+1 减 x，包含 LCA 与两端。演示 n,q≤50000、| 初值 |≤10^12、0≤x≤10^6，全部中间值用 long long。每次更新 $O(\log^2 n)$、点查 O(log n)、空间 O(n)。原 HDU3966 题面未核验。

所需模板：HLD（第 83 页，E.1.1 节）、Fenwick（第 68 页，C.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://github.com/kuangbin/ACM-ICPC

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    while (cin >> n >> m >> q)
    {
        vector<long long> a(n + 1);
        for (int u = 1; u <= n; u++) cin >> a[u];
        HLD h(n);
        for (int i = 0; i < m; i++)
        {
            int u, v;
            cin >> u >> v;
            h.add(u, v);
        }
        h.build();
        auto bit = Fenwick<long long>(n);
        while (q--)
        {
            char op;
            int u;
            cin >> op >> u;
            if (op == 'Q') cout << a[u] + bit.sum(h.dfn[u]) << '\n';
            else
            {
                int v;
                long long x;
                cin >> v >> x;
                if (op == 'D') x = -x;
                h.path(u, v, [&](int l, int r)
                {
                    bit.add(l, x);
                    bit.add(r + 1, -x);
                });
            }
        }
    }
}
```

应用题补充：SPOJ QTREE / 单边修改与路径最大边权

题意：多测 t≤20、n≤10000。输入边按 1..n−1 编号；CHANGE i x 修改第 i 条边，QUERY u v 查路径最大边权，DONE 结束本组。把边存到较深端点的 dfn−1；path 的 edge=true 排除 LCA，闭 [l,r] 转为 segtree 的半开 [l−1,r)。空路径约定输出 0，非空路径支持 signed64 负权及最小值；题面仅写 c≤10^6，未写修改值下界。预处理 O(n)、修改 O(log n)、查询 O($\log^2$ n)、空间 O(n)，递归 DFS。

所需模板：HLD（第 83 页，E.1.1 节）、segtree（第 75 页，D.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.spoj.com/problems/QTREE/

```
auto op(long long x, long long y)
{
    return max(x, y);
}

auto e()
{
    return numeric_limits<long long>::lowest();
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        HLD h(n);
        vector<int> a(n), b(n), child(n);
        vector<long long> w(n), base(n, e());
        for (int i = 1; i < n; i++)
        {
            cin >> a[i] >> b[i] >> w[i];
            h.add(a[i], b[i]);
        }
        h.build();
        for (int i = 1; i < n; i++)
        {
            child[i] = h.dep[a[i]] > h.dep[b[i]] ? a[i] : b[i];
            base[h.dfn[child[i]] - 1] = w[i];
        }
        auto seg = segtree<long long, op, e>(base);
        string cmd;
        while (cin >> cmd && cmd != "DONE")
        {
            int u;
            cin >> u;
            if (cmd == "CHANGE")
            {
                long long x;
                cin >> x;
                seg.set(h.dfn[child[u]] - 1, x);
            }
            else
            {
                int v;
                cin >> v;
                long long ans = e();
                h.path(u, v, [&](int l, int r)
                {
                    ans = max(ans, seg.prod(l - 1, r));
                }, true);
                cout << (u == v ? 0 : ans) << '\n';
            }
        }
    }
}
```

接口用法演示：树边翻转与奇路径计数（kuangbin HDU5039 模型）

演示约定：来源协议：T 组，每组 n、n 个不同名字、n-1 条 (名字, 名字,0/1) 边、q 条操作；Q 开头查询，其余操作接 1-based 输入边号并翻转该边。输出 Case #k: 及每次有向端点对数量，故答案 2*zero*one；无向点对则不乘 2。固定根 1，根路径异或值不同的端点间为奇路径；翻边即翻较深端点整棵子树，HLD 闭区间转 lazy_segtree 半开 [dfn-1,dfn-1+siz)。树上预处理 O(n)，名字 map 映射另计字符串比较和字典查询；改边 O(log n)，查询 O(1)，空间 O(n)，DFS 递归。官方 HDU 题面和限制未核验，只作来源模型/API 用法。

所需模板：HLD（第 83 页，E.1.1 节）、lazy_segtree（第 70 页，D.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：https://acm.hdu.edu.cn/showproblem.php?pid=5039

```
using S = pair<int, int>;

S op(S a, S b)
{
    return {a.first + b.first, a.second + b.second};
}

S e()
{
    return {0, 0};
}

S mapping(bool flip, S a)
{
    if (flip) swap(a.first, a.second);
    return a;
}

bool composition(bool f, bool g)
{
    return f ^ g;
}

bool id()
{
    return false;
}
```

使用入口（接上页配置函数）：

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    for (int tc = 1; tc <= tests; tc++)
    {
        int n;
        cin >> n;
        HLD h(n);
        map<string, int> names;
        for (int u = 1; u <= n; u++)
        {
            string name;
            cin >> name;
            names[name] = u;
        }
        vector<int> u(n), v(n), bit(n), child(n), parity(n + 1);
        for (int i = 1; i < n; i++)
        {
            string a, b;
            cin >> a >> b >> bit[i];
            u[i] = names.at(a);
            v[i] = names.at(b);
            h.add(u[i], v[i]);
        }
        h.build();
        for (int i = 1; i < n; i++)
        {
            child[i] = h.fa[u[i]] == v[i] ? u[i] : v[i];
            parity[child[i]] = bit[i];
        }
        vector<S> values(n);
        for (int i = 1; i <= n; i++)
        {
            int x = h.rk[i];
            if (x != 1) parity[x] ^= parity[h.fa[x]];
            values[i - 1] = {parity[x] == 0, parity[x] == 1};
        }
        lazy_segtree<S, bool, op, e, mapping, composition, id> seg(values);
```

继续处理操作（接上页同一 main）：

```
        int q;
        cin >> q;
        cout << "Case #" << tc << ":\n";
        while (q--)
        {
            string command;
            cin >> command;
            if (command[0] == 'Q')
            {
                auto [zero, one] = seg.all();
                cout << 2LL * zero * one << '\n';
            }
            else
            {
                int edge;
                cin >> edge;
                int x = child[edge];
                int l = h.dfn[x] - 1;
                seg.apply(l, l + h.siz[x], true);
            }
        }
    }
}
```

算法索引

AffineSegTree, 80

Binomial, 66

Chain3D, 34

digit_count, 10
Dominance3D, 32
Dominance4D, 27

ExactCover, 1

Fenwick, 68

grid_cycle, 12
grid_cycles, 16

HLD, 83

lazy_segtree, 70
Lucas, 64

MinimumCover, 5
Mo, 43
ModifiedMo, 45
ModInt, 62
monotone_dp_layer, 22
MonotoneHull, 20

OverallKth, 39

RollbackMo, 52

segtree, 75

TreeMo, 48

wqs_independent_set, 25

xor_hamming_pairs, 56

带修莫队, 59